

Asynchronous Programming

"ทำไมต้องเรียน Asynchronous Programming"

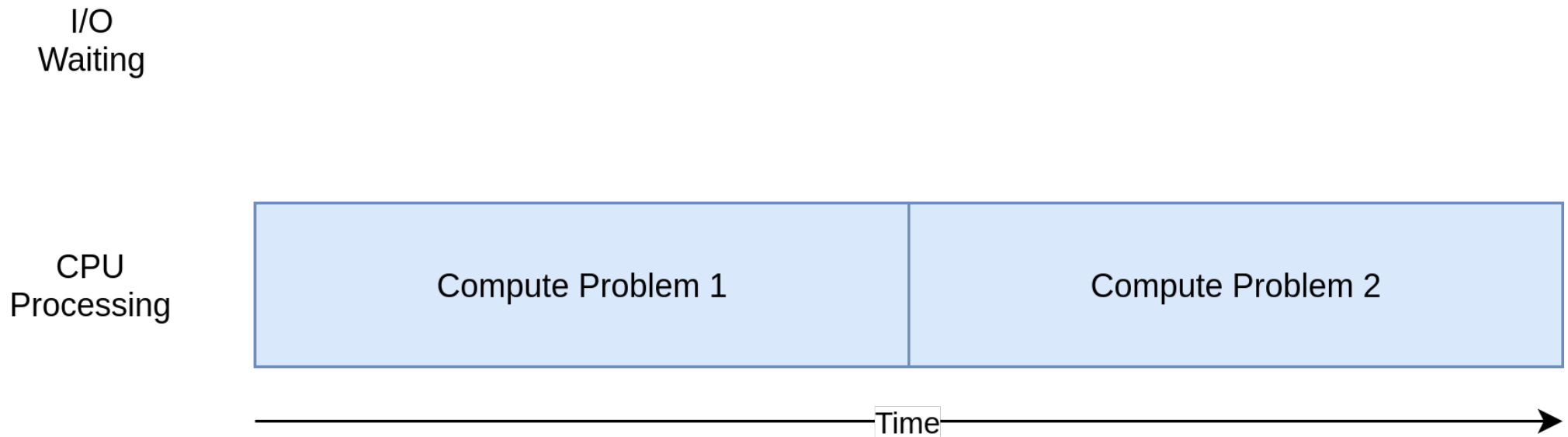
Asynchronous Programming Development Python

- Threading
- Multiprocessing
- Asynchronous
 - Event Loop
 - Task

CPU-Bound

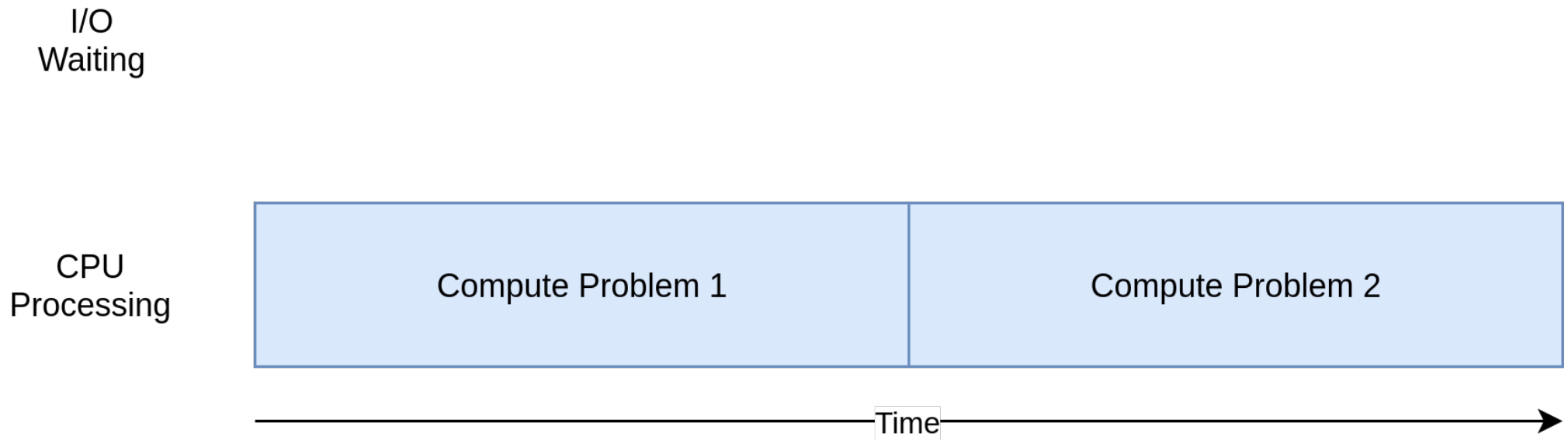
Single-Process Single-Thread Synchronous for CPU-Bound

- CPU-bound refers to a condition when the time for it to complete the task is determined principally by the **speed of the central processor**. The faster clock-rate CPU we have, the higher performance of our program will have.



CPU-Bound

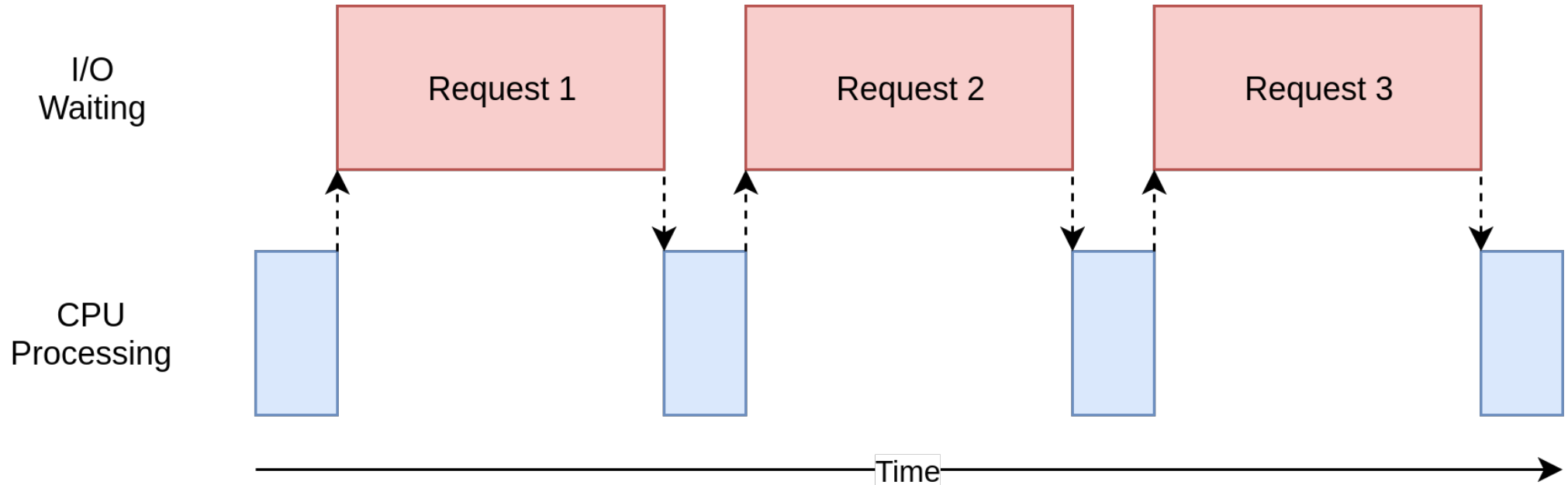
- Most of **single computer programs are CPU-bound**. For example, given a list of numbers, computing the sum of all the numbers in the list



I/O-Bound

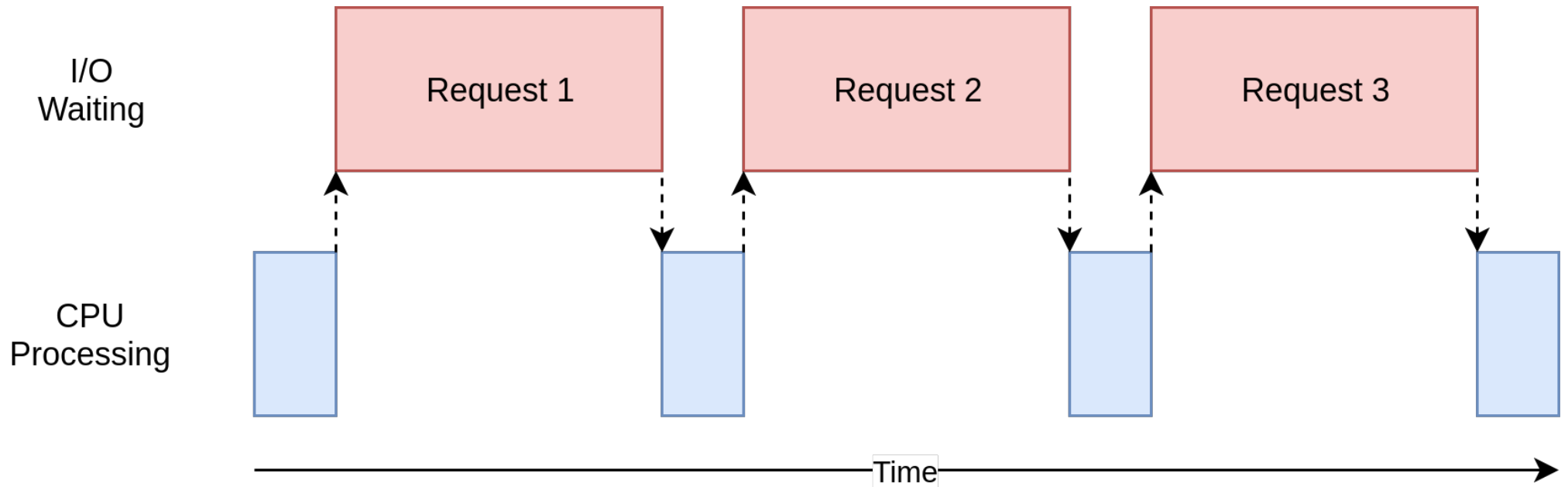
Single-Process Single-Thread Synchronous for I/O Bound

- I/O bound refers to a condition when the time it takes to complete a computation is determined principally by the **period spent waiting for input/output operations to be completed**



IO-Bound

- This is the opposite of a task being CPU bound. Increasing CPU clock-rate will not increase the performance of our program significantly. On the contrary, if we have **faster I/O**, such as **faster memory I/O**, **hard drive I/O**, **network I/O**, etc, the performance of our program will boost.



Process Thread

- Each program is a **process** and has **at least one thread** that **executes instructions** for that process
- When we run a Python script, it starts an instance of the **Python interpreter** that runs our code in the **main thread**. The main thread is the **default thread** of a Python process.
- When we create and run a new thread, Python will **make system calls on the underlying operating system** and **request a new thread be created** and to **start running the new thread**

Thread

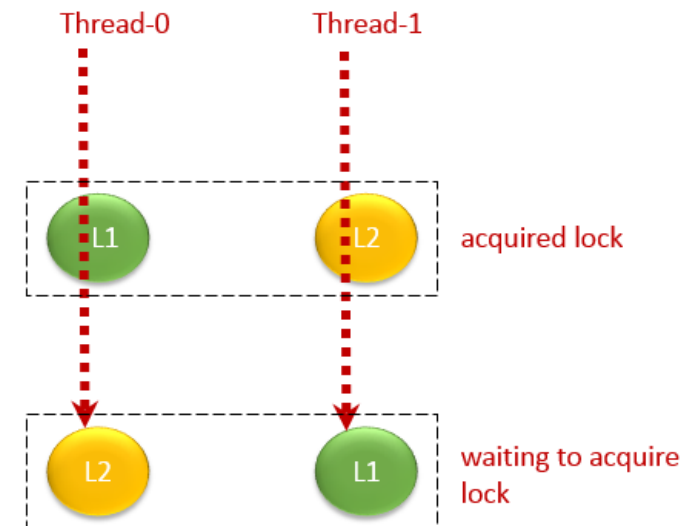
- Imagine a house (The **Process**). If you have one person living there, they must clean, cook, and work **sequentially**.
- Threads are multiple workers (people) in the same house. They **share** the kitchen and living room but can perform tasks simultaneously, making the household far **more efficient**.

RACE CONDITIONS

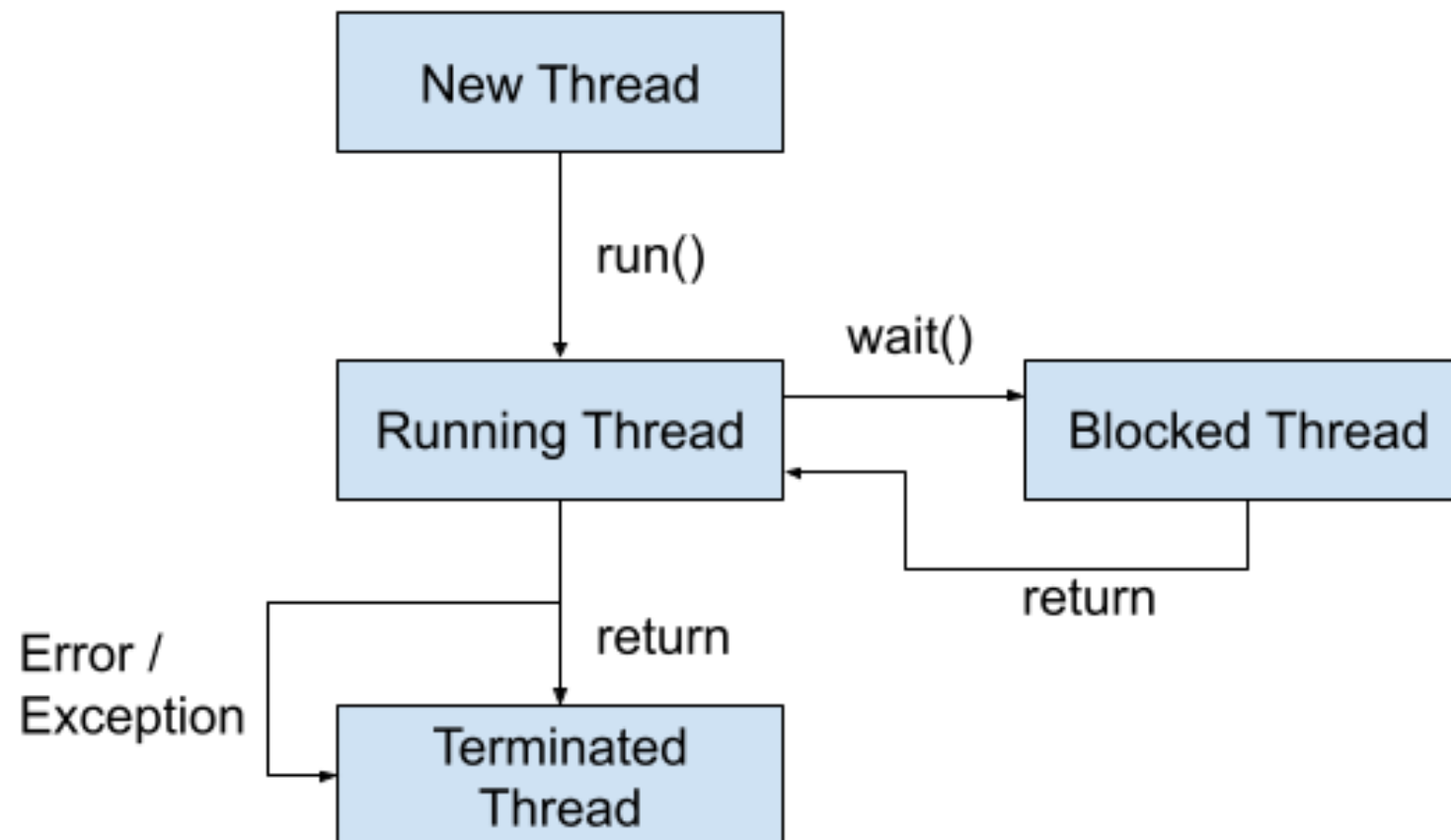
- When **two threads** try to **modify** the same **variable** at once, the final value depends on the timing of execution.
- Without locks, your data becomes inconsistent. It's like two people trying to grab the same single item from a shelf simultaneously.

THE DEADLOCK TRAP

- A Mutual Freeze
- Deadlock occurs when Thread **A** holds **Resource 1** and **waits** for **Resource 2**, while Thread **B** holds **Resource 2** and **waits** for **Resource 1**.
- **Neither** can proceed. The program stops entirely. Prevention requires careful ordering of resource acquisition or timeout strategies.



Life-Cycle of a Python Thread



Process VS Thread

Process in Python

- **Global interpreter Lock (GIL)**
a mechanism used in computer-language **interpreters** to synchronize the execution of threads so that **only one native thread can execute at a time**
- Python uses GIL
a single-process Python program could only **use one native thread during execution**. That means single-process Python program could not utilize CPU more than 100%
- C/C++
for **a single-process multi-thread C/C++ program**, it could utilize **many CPU cores** and many native threads, and the CPU utilization could be greater than 100%

GIL (Global Interpreter Lock)

What is the GIL?

- A strict "**traffic controller**" inside the standard Python interpreter (CPython).
- It restricts Python to executing bytecode on only one thread at a time, **regardless of how many CPU cores your computer has.**

GIL (Global Interpreter Lock)

Impact on CPU-Bound Tasks (Heavy Calculations)

- **Multi-threading** does **not** improve speed for **heavy mathematical** or data processing tasks.
- Threads **cannot** run in true **physical parallelism** because they must constantly wait in line to acquire the **GIL**.

GIL (Global Interpreter Lock)

The Exception: I/O-Bound Tasks (Networking & Storage)

- The Python interpreter **automatically releases** the **GIL** during **waiting** periods (e.g., waiting for an HTTPS request via **httpx** or **reading** a **database**).
 - While Thread 1 is blocked **waiting** for a network response,
 - Thread 2 immediately **takes** the lock and executes, achieving highly effective concurrency.

Concurrency in I/O Operations

THE PERFORMANCE EXCEPTION

- Efficient Waiting
 - The standard GIL bottleneck is bypassed during I/O-bound operations (Networking & Storage).
 - **Automatic Release:** The interpreter releases the GIL when a thread is waiting for a response (e.g., httpx requests).
 - **Parallel Execution:** While Thread 1 waits for the network, Thread 2 immediately takes the lock and executes its code.
 - **Ideal Use Cases:** Highly effective for web scraping, database queries, and microservices.

Multi-Processing with CPUs

Solution 1: Multi-processing (multiprocessing module)

- **How it works:** Instead of creating multiple threads within the same process, it spawns completely independent **Processes**.
- **The Secret:** Each new process gets its own Python interpreter and its **own dedicated GIL**.
- **Result:** The Operating System (OS) can distribute these processes across different CPU cores, achieving **100% True Physical Parallelism**.

Multi-Processing with CPUs

Solution 2: Third-Party C/C++ Extension Libraries

- **How it works:** Utilize high-performance libraries like **NumPy**, **Pandas**, **SciPy**, or **OpenCV** for heavy computations.
- **The Secret:** The core engine of these libraries is written in C/C++. When executing heavy operations (e.g., matrix multiplication), they execute a low-level command to **temporarily release the GIL**.
- **Result:** The underlying C code runs completely unhindered across **multiple CPU cores** simultaneously, returning the final **results** back to Python **once** finished.

Process in Python

- Therefore, for a CPU-bound task in Python, we would have to write **multi-process** Python program to maximize its performance

Thread in Python

- **A single-process Python** could **only use one CPU native thread**
- No matter how many threads were used in a single-process Python program
- **A single-process multi-thread Python** program could only achieve at most 100% CPU utilization
- However, this does not mean multi-thread is useless in Python. For a I/O-bound task in Python, **multi-thread** could be used to improve the program performance

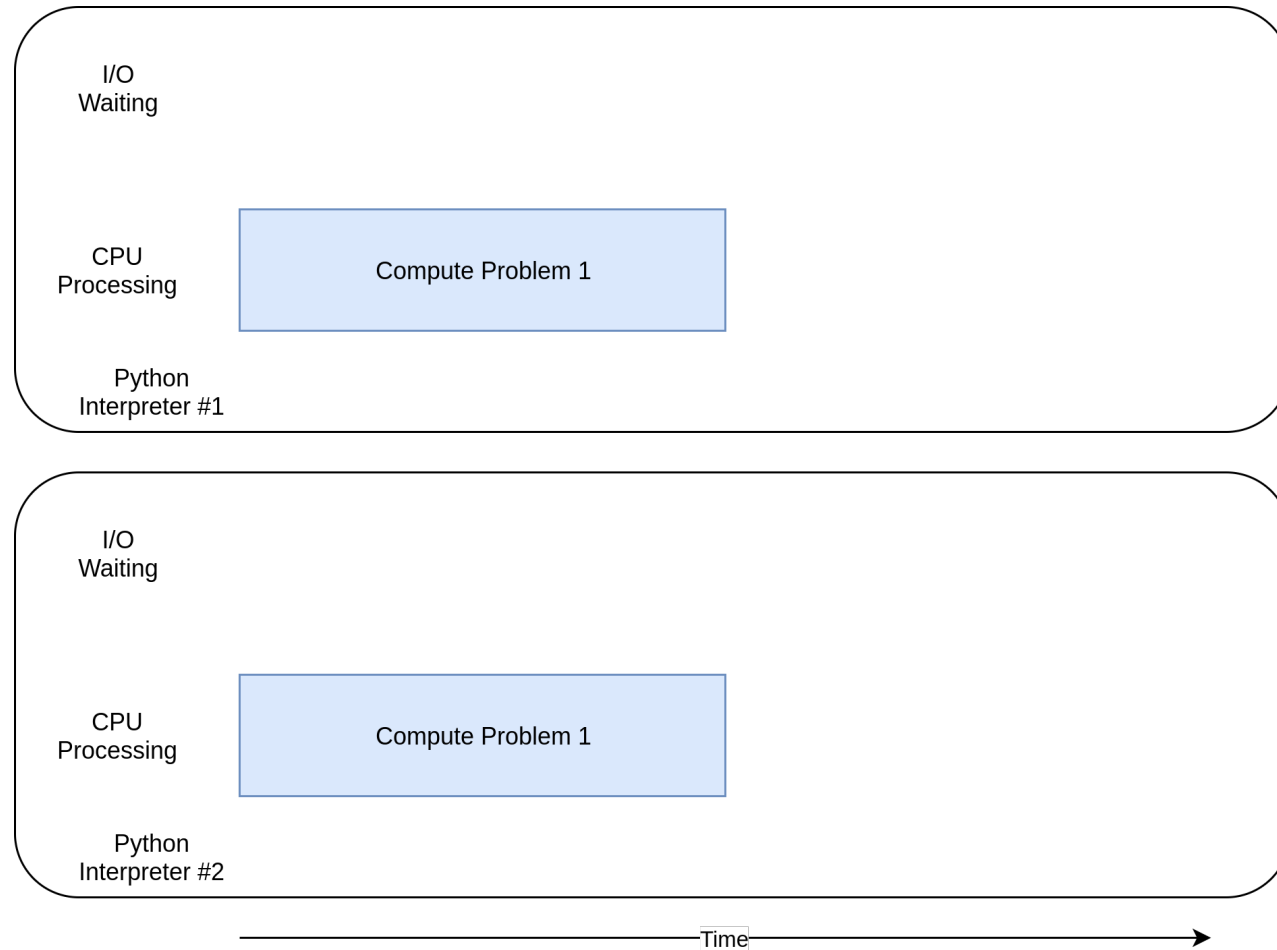
Multiprocessing VS Threading VS AsyncIO

Multiprocessing

- Using Python **multiprocessing**, we are able to run a Python using multiple processes
- A multi-process Python program could fully utilize all the CPU **cores** and **native threads** available, **by creating multiple Python interpreters on many native threads**
- All the **processes** are **independent** to **each other**, and they **don't share memory**
- To do **collaborative tasks in Python using multiprocessing**, it **requires** to use the **API provided** the operating system

Multiprocessing

Multi-Process for CPU-Bound



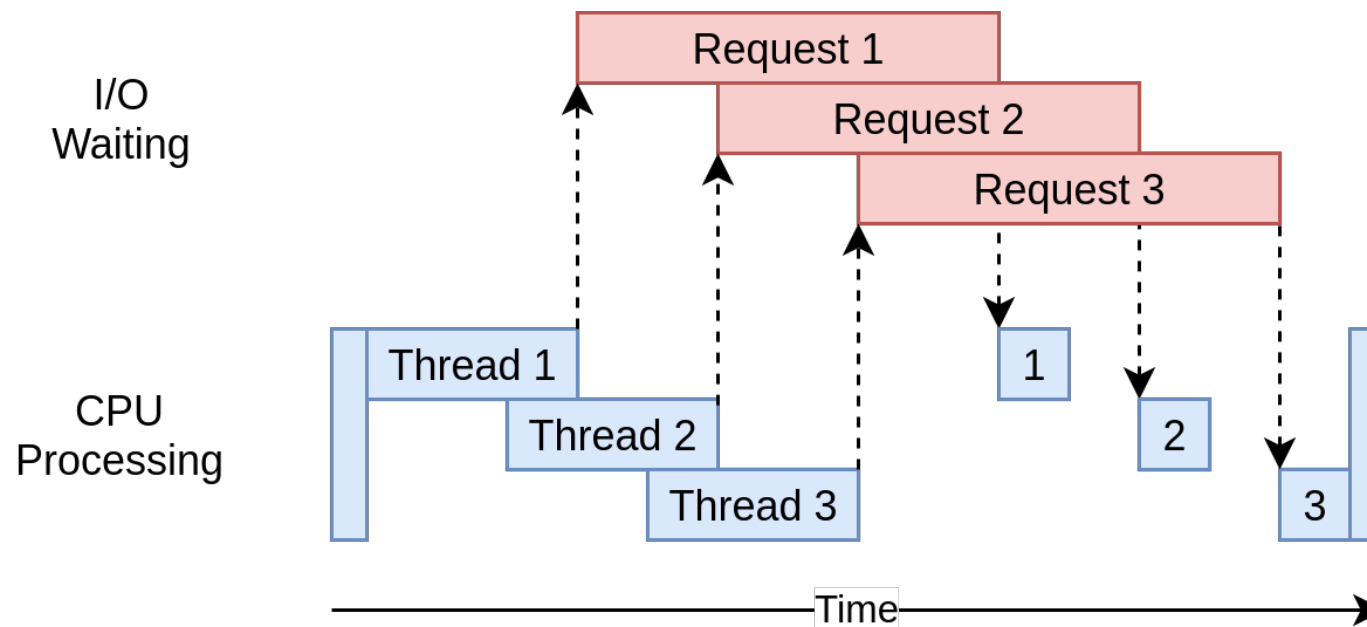
Threading

- Make better use of the **CPU sitting idle** when **waiting for the I/O**
- By **overlapping the waiting time for requests**, we are able to improve the performance
- All the **threads share the same memory**
- Have to be **careful** and use **locks** when **necessary**
- Lock and unlock make sure that **only one thread** could **write to memory** at one time, but this will also introduce some overhead
- The number of native threads in **CPU core is usually 2 nowadays**, but the number of **threads in a single-process Python program** could be much **larger than 2**

Threading

Single-Process Multi-Thread for I/O-Bound

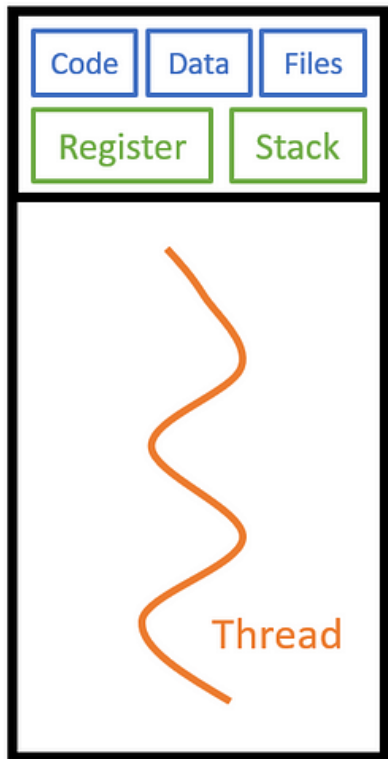
- For a I/O-bound task in Python, threading could be a good library candidate to use to maximize the performance



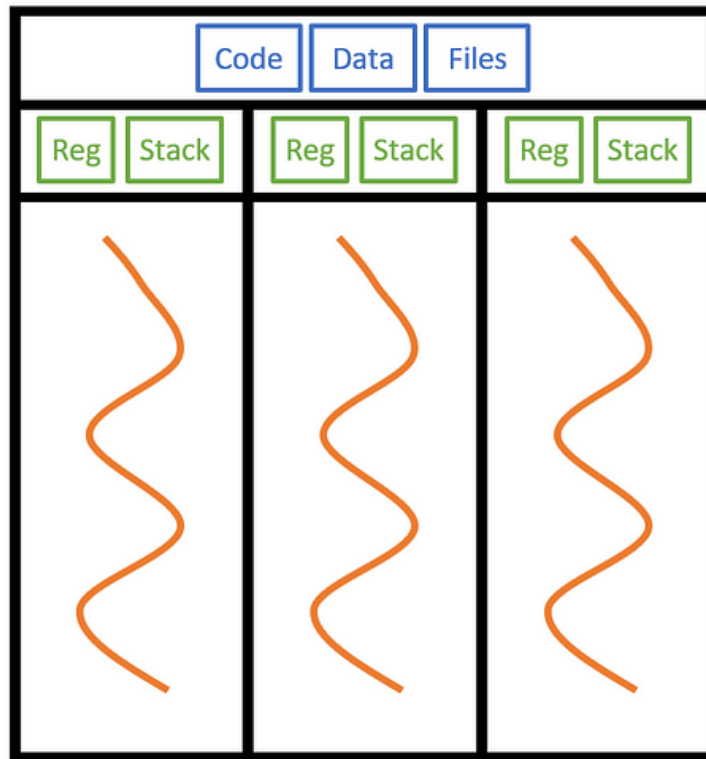
Threading

Single-Process Multi-Thread for I/O-Bound

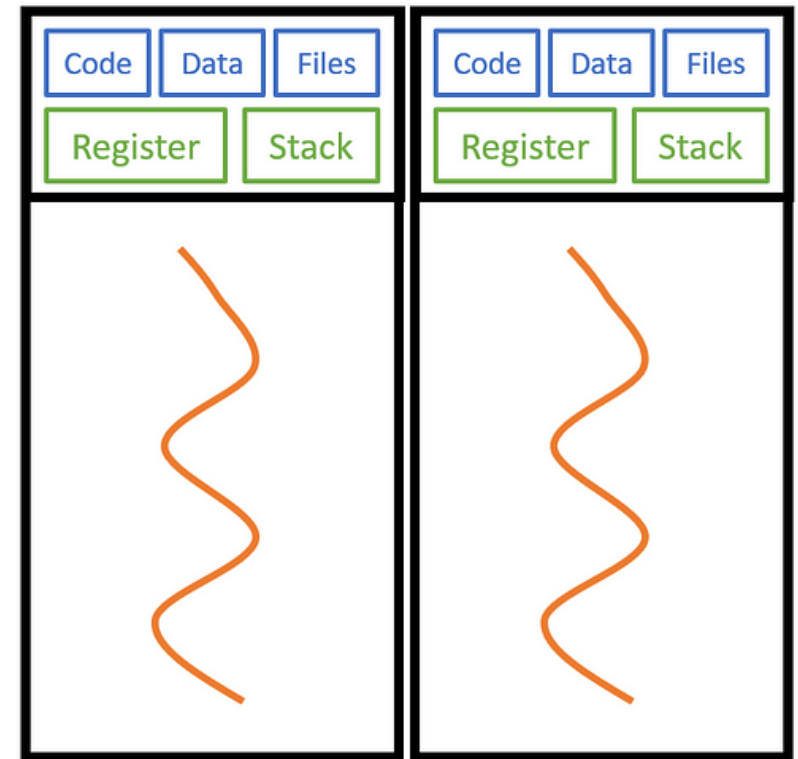
- All the threads are in a pool and there is an executer from the **operating system managing** the **threads deciding** who to run and when to run
- The operating system actually knows about each thread and can **interrupt** it at any time to start running a **different** thread
- This is called **pre-emptive** multitasking since the operating system can **pre-empt** your thread to make the switch



Single Processor Single Thread



Single Processor Multithread



Multiprocessing

Asynchronous I/O

If you know when to switch the tasks.

From Threading to Asyncio: The Evolution

- The Core Shift: **Single-Threaded Concurrency**
 - Unlike Threading, **Asyncio runs entirely on a SINGLE Thread.**
 - It uses an **Event Loop** to manage and schedule multiple tasks smoothly.

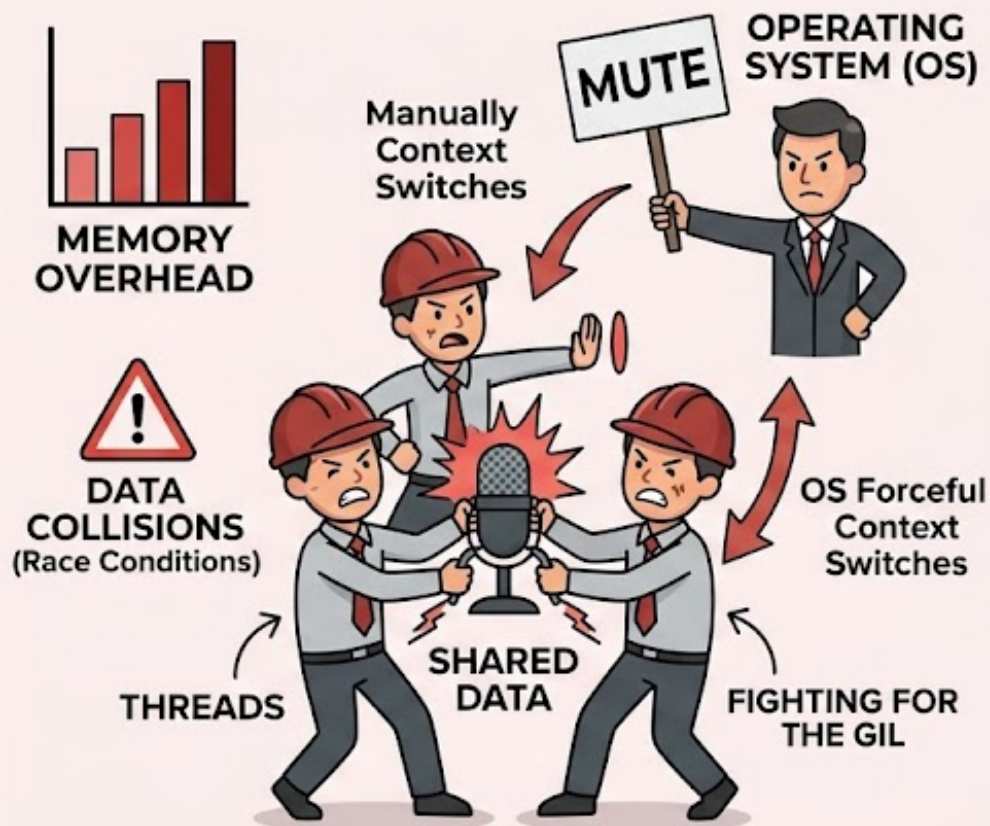
From Threading to Asyncio: The Evolution

- Cooperative vs. Preemptive
 - **Threading (Preemptive)**: The Operating System forcefully pauses and switches threads at any time (**high risk of Race Conditions**).
 - **Asyncio (Cooperative)**: Tasks voluntarily give up control (yield) using the await keyword only when they are waiting for I/O.

From Threading to Asyncio: The Evolution

- **Why** choose Asyncio over Threads?
 - **Ultra-Lightweight:** Creating a thread takes **megabytes** of memory; creating an async task takes **kilobytes**. You can easily run 10,000+ async tasks simultaneously.
 - **Data Safety:** Since only **one** thing executes at a time, you rarely need complex Locks or Mutexes to protect shared data.
 - **Lower Overhead:** No expensive OS context switching; execution control happens directly inside Python.

PYTHON THREADING: Preemptive & Risky



PYTHON ASYNCIO: Cooperative & Scalable



Asynchronous IO

- If you know when to switch the tasks.
- Python program using threading, it will really stay **idle** between the **request is sent** and the **result is returned**
- If somehow a thread could know the **time I/O request has been sent**, it could switch to do another task **without staying idle**, and one thread should be sufficient to manage all these tasks. Without the **thread management overhead**, the execution should be faster for a I/O-bound task
- Obviously, threading **could not do** it, but we have **asyncio**

Asynchronous IO

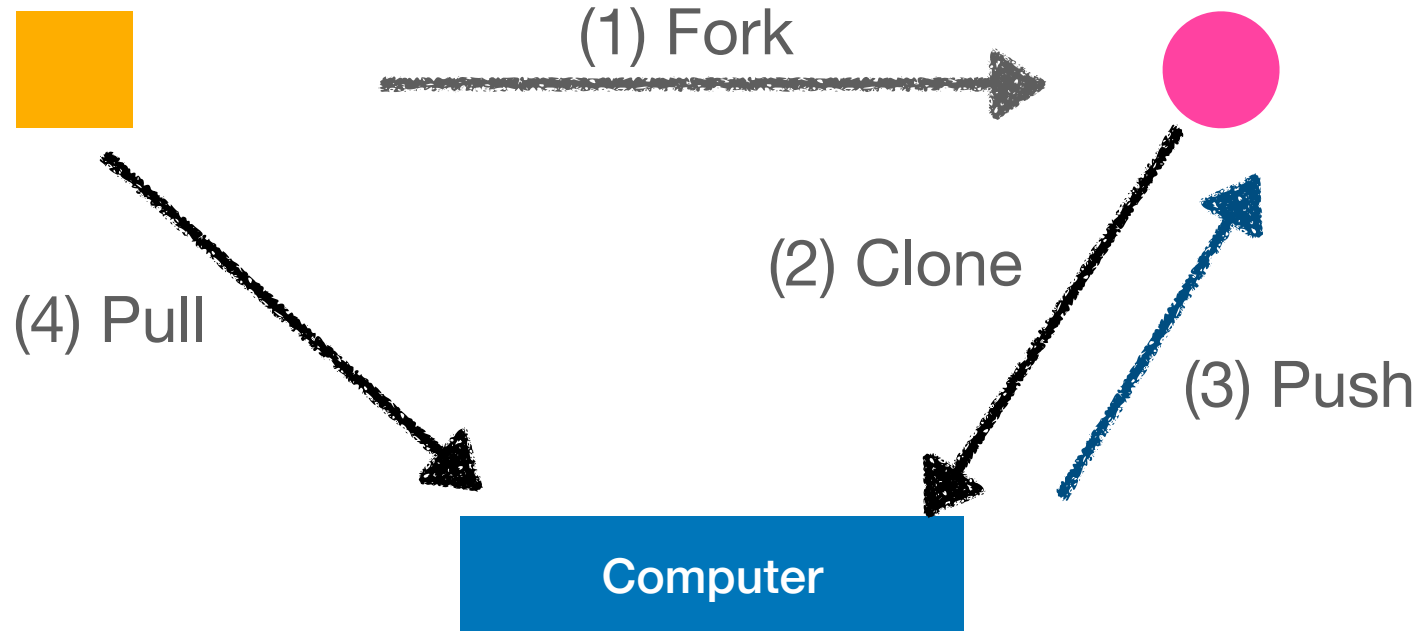
- Using Python asyncio, we are also able to make better use of the CPU sitting **idle** when **waiting** for the I/O, different to threading is that
- asyncio is **single-process** and **single-thread**
- There is an **event loop** in asyncio which routinely **measure the progress** of the tasks
- it would **schedule another task for execution**, therefore, **minimizing** the time **spent** on **waiting I/O**.
- called **cooperative** multitasking, The tasks must cooperate by **announcing when they are ready to be switched out**

Asynchronous IO

- asyncio is that the even loop **would not know what are the progresses** if we **don't tell it**. This requires some additional effort when we write the programs using asyncio.

<https://github.com/hanattaw/async-2026.git>

<https://github.com/students/async-2026.git>



Coding Workflow

- คิดแบบสถาปนิก (Top-Down): เห็นภาพปลายทาง รู้องค์ประกอบหลัก
- สร้างแบบช่างฝีมือ (Bottom-Up): ทำทีละชิ้นส่วนให้สมบูรณ์แบบและใช้งานได้จริง
- ตรวจสอบสม่ำเสมอ: ไม่ปล่อยให้ข้อผิดพลาดเล็กๆ ลุกลามจนแก้ไม่ได้ตอนภาพเสร็จแล้ว

การ coding ที่ดี ไม่ใช่นั่งมองหน้าจอว่างๆ แล้วพิมพ์ไปเรื่อยๆ

CODING WORKFLOW: จากภาพร่างสู่ความจริง

(From Sketch to Reality)



Synchronous coffee

```
1  from time import sleep, ctime, time
2
3  # ฟังก์ชันการชงกาแฟ
4  def make_coffee(customer_name):
5      sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
6
7  def main():
8      # กระบวนการชงกาแฟแต่ละลูกค้า
9      make_coffee('A')
10
11  # สั่งให้โปรแกรมทำงาน
12  if __name__ == "__main__":
13      main()
```


พฤติกรรมที่ 1

Synchronous (รอกกาแฟ 1 ตัว - บล็อกแถว)

พฤติกรรม: ทุกคนถูกแช่แข็ง ต้องรอคนหน้าขงเสร็จก่อนเท่านั้น

พฤติกรรมแบบซิงโครนัส (Synchronous): ลูกค้า A, B, C เดินมาต่อแถวที่ตู้กาแฟตัวอื่นๆ ไม่สามารถทำงานคันเวลาได้

- A** ลูกค้า A กดสั่ง: หน้าจอล็อกขึ้น "กำลังชงแก้ว A" ลูกค้า A ยืนรอหน้าตู้ขยับไปไหนไม่ได้
- B** ลูกค้า B ยืนรอคิว: ไม่สามารถกดปุ่มสั่งหรือทำอะไรได้เลย ต้องรอจนกว่า A จะได้กาแฟและเดินออกไป
- C** ลูกค้า C รอท้ายสุด: ต้องผ่านกระบวนการรอคูลองสอง เสียเวลาสะสมเป็นทวีคูณ

จำลองพฤติกรรม: Synchronous (1 ตัว)

ตู้ว่าง / พร้อมใช้งาน

คิวรอประมวลผล (Queue)



ทุกคนเข้าแถวรอตามลำดับ

COFFEE MACHINE (1)

[ตู้ว่าง]
กรุณากดปุ่มชง

ไม่มีแก้ว

▶ เริ่มรันแบบลำดับทีละคน

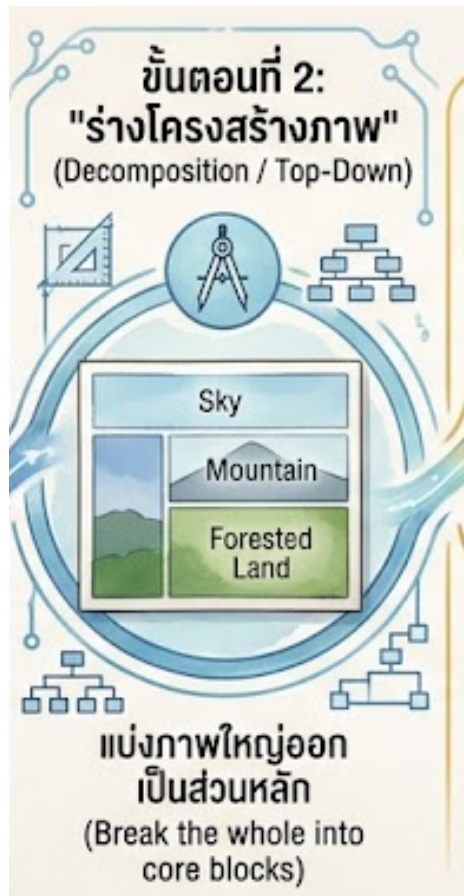
ขั้นตอนที่ 1: มองเห็นภาพรวม (Requirement)



- “เป้าหมาย”
- ต้องการคำนวณและแสดงผลให้เห็นว่า ลูกค้าแต่ละคนเข้ามาสั่งกาแฟแล้วต้องใช้เวลารอกี่วินาที จนกระทั่ง "คนสุดท้าย (ลูกค้า C) ได้รับกาแฟในวินาทีที่เท่าไหร่" โดยแสดงสถานะการทำงานออกมาเป็นลำดับเชิงโครนัส

ขั้นตอนที่ 2: ร่างโครงสร้างภาพ (Decomposition / Top-Down)

แตก โปรแกรมออกเป็นส่วนย่อย ๆ



- ส่วนข้อมูล => รายชื่อลูกค้า
- กระบวนการทำงานของตู้กาแฟ (Process) การชงกาแฟ 1 แก้วที่ใช้เวลา 1 วินาที
- ส่วนควบคุมคิวและการคำนวณเวลา ให้ทำงานที่ละคนตามลำดับ สะสมเวลาไปเรื่อย ๆ
- ส่วนแสดงผลลัพธ์ โดยแสดงว่า ทั้ง 3 คน

ขั้นตอนที่ 3: ลงรายละเอียดทีละส่วน (Implementation / Bottom-Up)



- ส่วนข้อมูล => รายชื่อลูกค้า
- กระบวนการทำงานของตู้กาแฟ (Process) การชงกาแฟ 1 แก้วที่ใช้เวลา 1 วินาที
- ส่วนควบคุมคิวและการคำนวณเวลา ให้ทำงานทีละคนตามลำดับ สะสมเวลาไปเรื่อย ๆ
- ส่วนแสดงผลลัพธ์

```
# คิวลูกค้า
queue = ['A', 'B', 'C']
```

```
# ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
def make_coffee(customer_name):
    print(f"{ctime()} กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
    sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
    print(f"{ctime()} ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
```

```
print(f"{ctime()} === เริ่มระบบจำลองตู้กาแฟ Synchronous ===")
start_time = time()
```

```
# ลูปทำงานตามลำดับ Synchronous
for customer in queue:
    # กระบวนการชงกาแฟแต่ละลูกค้า
    make_coffee(customer)
```

```
duration = time() - start_time # คำนวณเวลาที่ใช้
print(f"{ctime()} ลูกค้า {len(queue)} คน ใช้เวลา: {duration:0.2f} วินาที")
```

ขั้นตอนที่ 4: ตรวจสอบ (Testing / Debugging)



- ลองให้ลูกค้า 1 คน กด กาแฟ

```
1  from time import sleep, ctime, time
2
3  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
4  def make_coffee(customer_name):
5      print(f"{ctime()} กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
6      sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
7      print(f"{ctime()} ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
8
9  # สั่งให้โปรแกรมทำงาน
10 if __name__ == "__main__":
11     make_coffee('A')
```


ขั้นตอนที่ 5: "ประกอบภาพและเก็บรายละเอียด" (Integration & Refactoring)



- นำชิ้นส่วนต่างๆ มาร้อยเรียงกันในฟังก์ชันหลัก (main) เพื่อคำนวณเวลารวมทั้งหมด และจับตาดูว่าลูกค้าคนสุดท้าย (C) จะได้กาแฟเมื่อไหร่

```
1 from time import sleep, ctime, time
2
3 # ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
4 def make_coffee(customer_name):
5     print(f"{ctime()} กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
6     sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
7     print(f"{ctime()} ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
8
9 def main():
10     # คิวลูกค้า
11     queue = ['A', 'B', 'C']
12     print(f"{ctime()} === เริ่มระบบจำลองตู้กาแฟ Synchronous ===")
13     start_time = time()
14
15     # ลูปทำงานตามลำดับ Synchronous
16     for customer in queue:
17         # กระบวนการชงกาแฟแต่ละลูกค้า
18         make_coffee(customer)
19
20     duration = time() - start_time # คำนวณเวลาที่ใช้
21     print(f"{ctime()} ลูกค้า {len(queue)} คน ใช้เวลา: {duration:0.2f} วินาที")
22
23 # สั่งให้โปรแกรมทำงาน
24 if __name__ == "__main__":
25     main()
```

Multi-Thread Coffee

- เมื่อเราเปลี่ยนพฤติกรรมจาก Synchronous (วาดรูปทีละส่วน/ชงกาแฟทีละแก้วแบบต่อคิว) มาเป็น Multi-threading เปรียบเสมือนเรา "จ้างช่างวาดภาพหลายคนมาช่วยกันลงสีพร้อมกัน" หรือในโจทย์นี้คือ "ตู้กาแฟชงกาแฟได้ 3 แก้วพร้อมกัน" เพื่อให้ลูกค้า A, B, และ C สามารถกดสั่งและได้กาแฟพร้อมๆ กัน โดยไม่ต้องยืนรอคนข้างหน้า

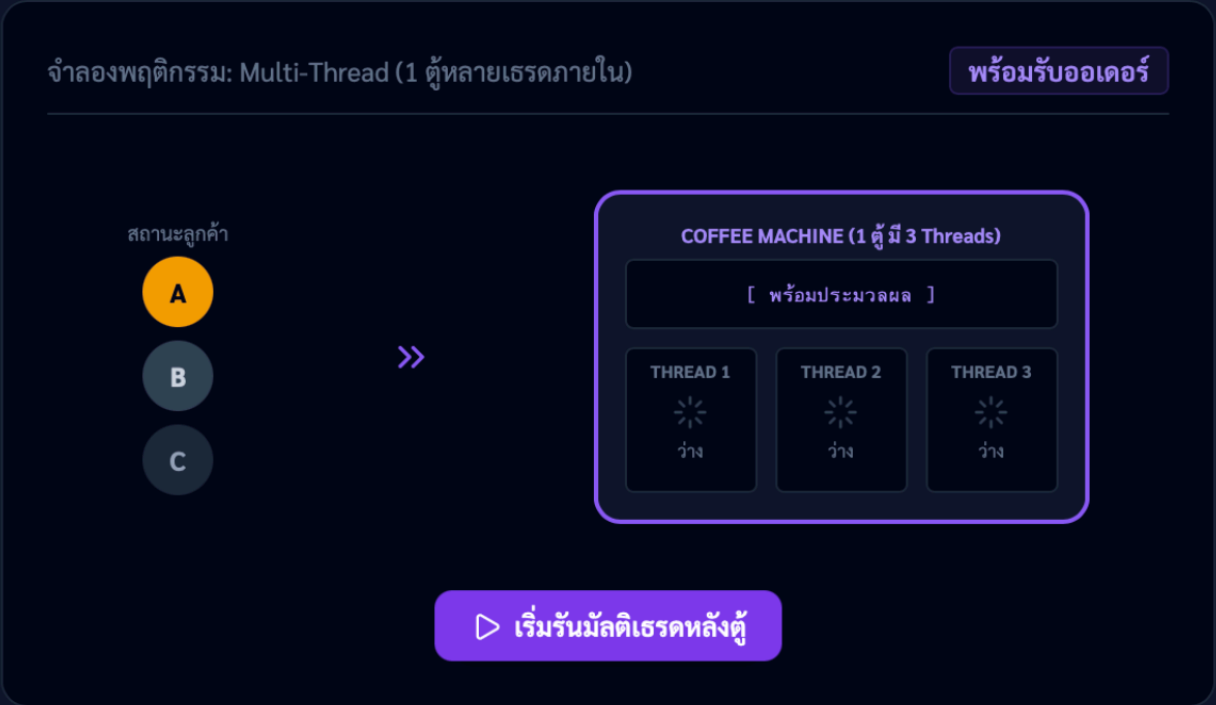
```
1  # running a function in another thread
2  from time import sleep, ctime
3  from threading import Thread
4
5  # a custom function that blocks for a moment
6  def task():
7      # block for a moment
8      sleep(1)
9      # display a message
10     print(f'{ctime()} This is from another thread')
11
12     # create a thread
13     thread = Thread(target=task)
14     # run the thread
15     thread.start()
16     # wait for the thread to finish
17     print(f'{ctime()} Waiting for the thread...')
18     thread.join()
```

Thread (รอกกาแฟ 1 ตู้ - ชงพร้อมกันในตู้เดียว)

พฤติกรรม: 1 ตู้ที่มีชุดหัวชงย่อย 3 ชุดทำงานแยกกันในตู้เดียว

พฤติกรรมแบบมัลติเธรด (Multi-Thread): เปรียบเหมือนตู้กาแฟสุดหรูที่มี 1 หน้าจอ แต่ด้านในแผงวงจรมี "แขนกล/หัวจ่าย 3 ชุด" คอยชงแบบขนาน

- 1 หน้าจอรับคำสั่งต่อเนื่อง: ลูกค้า A, B, C เข้ามากดสั่งอย่างรวดเร็ว (เหมือน Async)
- 2 กระจายงานเข้าเธรด: เครื่องจ่ายงาน A ให้เธรดที่ 1, งาน B ให้เธรดที่ 2, งาน C ให้เธรดที่ 3 ทำการบดและกลั่นพร้อมกันในตู้เดียว
- 3 จบงานเร็วขึ้น: แม้จะตู้เดียว แต่กาแฟ 3 แก้วชงไหลออกมาในห้วงจ่ายแก้วแทบจะพร้อมๆ กัน



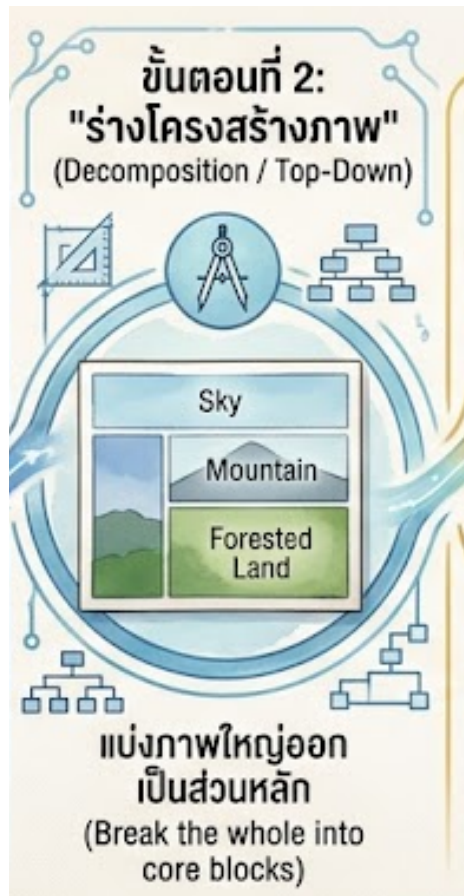
ขั้นตอนที่ 1: มองเห็นภาพรวม (Requirement)



- “เป้าหมาย”
- การคิดเชิงเปรียบเทียบ: จากเดิมที่เรามองภาพ "คนยืนต่อคิวรอใช้เครื่องกดกาแฟเดียวกัน" เปลี่ยนภาพรวมใหม่เป็น "เครื่องกดกาแฟที่มีลูกค้า 3 คน มาสั่งกาแฟพร้อมๆ กัน"
- เป้าหมายของระบบ Threading: ทุกคน (A, B, C) ต้องสามารถเริ่มสตาร์ทกระบวนการชงกาแฟได้ทันทีในวินาทีเดียวกัน โดยไม่มีใครมาบล็อกคิวใคร และเวลารวมทั้งหมดของระบบจะต้องลดลงเหลือเท่ากับการทำกาแฟเพียงแก้วเดียว (คือ 1 วินาที)

ขั้นตอนที่ 2: ร่างโครงสร้างภาพ (Decomposition / Top-Down)

แตก โปรแกรมออกเป็นส่วนย่อย ๆ



- ส่วนเตรียมผู้ทำงาน (Thread Creation): ต้อง มีรูปสำหรับสร้าง และแจกจ่ายงานให้แต่ละคน
- ส่วนสั่งเริ่ม (Thread Start): สั่งให้พนักงานทุกคนเริ่มทำงานพร้อมกันทันที
- ส่วนควบคุมการรอ (Thread Synchronization/Join): ต้องมีจุดรวม เพื่อให้โปรแกรมหลัก (Main) นั่งรอจนกระทั่งลูกค้าทุกคนทำงานได้กาแฟครบ ถึงจะสรุปเวลาปิดงานได้

ขั้นตอนที่ 3: ลงรายละเอียดทีละส่วน (Implementation / Bottom-Up)



- ส่วนที่ 1: เตรียมฟังก์ชันงาน (Worker) (ใช้ของเดิมได้เลย เพราะฟังก์ชันชงกาแฟทำงานอิสระอยู่แล้ว)
- ส่วนที่ 2: ร่างคำสั่งสร้างและสั่งวิ่ง Thread
- ส่วนที่ 3: ร่างคำสั่งรอผลลัพธ์ (Join)

```
# ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
def make_coffee(customer_name):
    print(f"{ctime()} กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
    sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
    print(f"{ctime()} ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
```

```
# สร้าง Thread ใหม่ โดยระบุให้ไปวิ่งที่ฟังก์ชัน make_coffee และส่งชื่อลูกค้าไปเป็น argum
t = threading.Thread(target=make_coffee, args=(customer,))
threads.append(t)
t.start() # สั่งให้ Thread เริ่มทำงานทันที (ไม่ต้องรอให้รอบลูปถัดไปเสร็จ)
```

```
# รอให้ "ช่างทุกคน" ชงกาแฟเสร็จครบหมดก่อน ถึงจะไปสเด็ดปิดไป
for t in threads:
    t.join()
```

```
duration = time() - start_time # ส่วนของเวลาที่ใช้จริง
```

ขั้นตอนที่ 4: ตรวจสอบ (Testing / Debugging)



- การทำ Threading มักจะเจอปัญหาเรื่อง "ข้อมูลตีสัน" หรือ "ลำดับการแสดงผลสลับกัน" ในขั้นตอนนี้เราจึงต้องทดสอบดูว่า
- ลูกค้า A, B, C เริ่มต้นในวินาทีเดียวกันจริงไหม? (ดูจาก Timestamp ของ `ctime()`)
- ข้อความที่ Print ออกมาบนหน้าจอ มีการซ้อนทับหรือเบียดกันจนอ่านไม่รู้เรื่องหรือไม่?
- หากส่งคิวเข้าไปเยอะๆ ระบบยังทำงานได้ถูกต้องและไม่เกิดอาการโปรแกรมค้าง (Deadlock) หรือไม่?

```
Sun Jun 21 22:04:29 2026 กำลังซิงกาแฟให้ ลูกค้า A...
Sun Jun 21 22:04:29 2026 กำลังซิงกาแฟให้ ลูกค้า B...
Sun Jun 21 22:04:29 2026 กำลังซิงกาแฟให้ ลูกค้า C...
Sun Jun 21 22:04:30 2026 ลูกค้า B: ได้รับกาแฟแล้ว!
Sun Jun 21 22:04:30 2026 ลูกค้า A: ได้รับกาแฟแล้ว!
Sun Jun 21 22:04:30 2026 ลูกค้า C: ได้รับกาแฟแล้ว!
Sun Jun 21 22:04:30 2026 ลูกค้า 3 คน ได้รับกาแฟครบแล้ว! ใช้เวลารวมทั้งหมด: 1.01 วินาที
```


ขั้นตอนที่ 5: "ประกอบภาพและเก็บรายละเอียด" (Integration & Refactoring)



- นำชิ้นส่วนการสร้าง Thread, การสั่ง Start, และการสั่ง Join มาร้อยรวมกันในฟังก์ชัน main() แล้วเกลี่ยโค้ดให้สะอาดเรียบร้อย โดยใช้ List มาช่วยจัดเก็บกลุ่มก้อนของ Thread

```
1 from time import sleep, ctime, time
2 import threading
3
4 # ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
5 def make_coffee(customer_name):
6     print(f"{ctime()} กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
7     sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
8     print(f"{ctime()} ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
9
10 def main():
11     # คิวลูกค้า
12     queue = ['A', 'B', 'C']
13     print(f"{ctime()} === เริ่มระบบจำลองตู้กาแฟแบบ Multi-threading ===")
14     start_time = time()
15
16     # รายชื่อที่จะเอาไว้เก็บ Thread ของแต่ละคน
17     threads = []
18
19     # สร้าง "(Thread)" แยกให้ลูกค้าแต่ละคนทำงานพร้อมกัน
20     for customer in queue:
21         # สร้าง Thread ใหม่ โดยระบุให้ไปวิ่งที่ฟังก์ชัน make_coffee และส่งชื่อลูกค้าไปเป็น argument
22         t = threading.Thread(target=make_coffee, args=(customer,))
23         threads.append(t)
24         t.start() # สั่งให้ Thread เริ่มทำงานทันที (ไม่ต้องรอให้รอบลูปถัดไปเสร็จ)
25
26     # รอให้ "ช่างทุกคน" ชงกาแฟเสร็จครบหมดก่อน ถึงจะไปสแต็ปถัดไป
27     for t in threads:
28         t.join()
29
30     duration = time() - start_time # คำนวณเวลารวมที่ใช้จริง
31     print(f"{ctime()} ลูกค้า {len(queue)} คน ได้รับกาแฟครบแล้ว! ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
32
33 # สั่งให้โปรแกรมทำงาน
34 if __name__ == "__main__":
35     main()
```

Multi-Processing Coffee

- Multi-processing ในเชิงเปรียบเทียบกับการวาดภาพ จะไม่ใช่แค่การเพิ่มช่างวาดภาพมาแชร์สตูดิโอเดียวกันแต่มันคือการ "เปิดสตูดิโอใหม่แยกออกจากกันโดยสิ้นเชิง" มีผืนผ้าใบเป็นของตัวเอง มีสีเป็นของตัวเอง ไม่แย่งพื้นที่หรือทรัพยากรกัน
- กรณีการชงกาแฟคือ มี 3 ตู้ สำหรับลูกค้า 3 คน ที่แต่ละตู้ไม่แย่งทรัพยากรกัน

```
1  # Multiprocessing task
2  from time import sleep, ctime, time
3  import multiprocessing
4
5  # a custom function
6  def task():
7      sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที (แทน 1 นาที)
8      print(f"{ctime()} This is from another process")
9
10 if __name__=="__main__":
11     # Begin of main thread
12     start_time = time()
13     p = multiprocessing.Process(target=task)
14     # starting processes
15     p.start()
16
17     # wait until processes are finished
18     p.join()
```

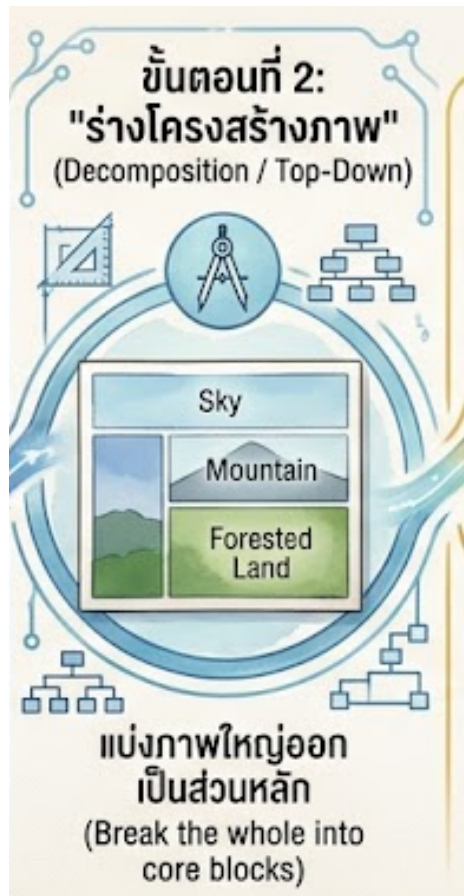
ขั้นตอนที่ 1: มองเห็นภาพรวม (Requirement)



- “เป้าหมาย”
- การคิดเชิงเปรียบเทียบ: เปลี่ยนจาก “ลูกค้า 3 คนรุมสั่งกาแฟที่เครื่องเดียว” เป็น “ตั้งตู้ชงกาแฟแยกกัน 3 ตู้” แต่ละตู้ทำงานอิสระต่อกันอย่างสมบูรณ์
- เป้าหมายของระบบ: รันงานแยกจากกันเด็ดขาดเพื่อประสิทธิภาพสูงสุด และทำให้ได้ใกล้เคียง 1 วินาทีเท่าเดิม

ขั้นตอนที่ 2: ร่างโครงสร้างภาพ (Decomposition / Top-Down)

แตก โปรแกรมออกเป็นส่วนย่อย ๆ



- โครงสร้างของ Multi-processing จะคล้ายกับ Threading ในแง่ Lifecycle
- จุดที่ต้องคำนึงเพิ่มคือ "โครงสร้างความปลอดภัยของ Main Process" เพราะบนระบบปฏิบัติการอย่าง Windows หรือ macOS การแตก Process ใหม่จำเป็นต้องรับภายใต้บล็อกคัมกัน if `__name__ == "__main__"`: เสมอ ไมอย่างนั้น โปรแกรมจะแตกตัวไม่สิ้นสุดจนค้าง
- ส่วนห่อหุ้มความปลอดภัย: กำหนดจุดเริ่มต้นหลัก (`__main__`)
- ส่วนกระจาย Process: วนลูปเพื่อสร้างหน่วยประมวลผลแยกอิสระ
- ส่วนสั่งทำงานและจตุรวมพล: สั่ง `.start()` และ `.join()` เหมือนเดิม

ขั้นตอนที่ 3: ลงรายละเอียดทีละส่วน (Implementation / Bottom-Up)



- ส่วนที่ 1: ฟังก์ชันการทำงาน (ใช้ฟังก์ชันเดิมได้เลย)
- ส่วนที่ 2: เปลี่ยนคำสั่งสร้างหน่วยประมวลผล

```
# ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
def make_coffee(customer_name):
    print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]  
กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
    sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที
    print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]  
ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
```

```
# 1. แยกกระบวนการทำงานแยกจากกันเป็นอิสระ (คนละตัว คนละ CPU Core)
for customer in queue:
    p = multiprocessing.Process(target=make_coffee, args=(customer,))
    processes.append(p)
    p.start() # สั่งเริ่มประมวลผลทันที

# 2. รอจนกระทั่งทุก Process ทำงานเสร็จครบหมด
for p in processes:
    p.join()
```

ขั้นตอนที่ 4: ตรวจสอบ (Testing / Debugging)



- การตรวจสอบใน Multi-processing ต้องระวังเรื่องสำคัญ คือ Process ID (PID) และหน่วยความจำที่แยกจากกัน:
- ตรวจสอบว่าใน Task Manager หรือ Activity Monitor มีกระบวนการ Python ก่อขึ้นขึ้นมาใหม่ 3 อันจริงไหม?
- ข้อมูลที่พิมพ์ออกมาทางหน้าจอต้องไม่ตีกัน (ซึ่งบางระบบปฏิบัติการอาจจะแสดงผลสลับบรรทัดกันเล็กน้อยเนื่องจากส่งข้อมูลมาจากคนละหน่วยความจำ)

```
Sun Jun 21 22:43:00 2026 === เริ่มระบบจำลองดูกาแฟแบบ Multi-processing ===
Sun Jun 21 22:43:00 2026 [Process ID: 17734] กำลังชงกาแฟให้ ลูกค้า A...
Sun Jun 21 22:43:00 2026 [Process ID: 17736] กำลังชงกาแฟให้ ลูกค้า C...
Sun Jun 21 22:43:00 2026 [Process ID: 17735] กำลังชงกาแฟให้ ลูกค้า B...
Sun Jun 21 22:43:01 2026 [Process ID: 17734] ลูกค้า A: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17736] ลูกค้า C: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17735] ลูกค้า B: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 สำเร็จ! ทุกคู่ทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด: 1.07 วินาที
```

ขั้นตอนที่ 5: "ประกอบภาพและเก็บรายละเอียด" (Integration & Refactoring)



- ประกอบชิ้นส่วน

```
1 from time import sleep, ctime, time
2 import multiprocessing
3
4 # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
5 def make_coffee(customer_name):
6     print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]
7     กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
8     sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที
9     print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]
10    ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
11
12 def main():
13     # คิวลูกค้า
14     queue = ['A', 'B', 'C']
15     print(f"{ctime()} === เริ่มระบบจำลองตู้กาแฟแบบ Multi-processing ===")
16     start_time = time()
17
18     processes = [] # กระดานเก็บรายชื่อหน่วยประมวลผล (Processes)
19
20     # 1. แดกกระบวนการทำงานแยกจากกันเป็นอิสระ (คนละตู้ คนละ CPU Core)
21     for customer in queue:
22         p = multiprocessing.Process(target=make_coffee, args=(customer,))
23         processes.append(p)
24         p.start() # สั่งเริ่มประมวลผลทันที
25
26     # 2. รอจนกระทั่งทุก Process ทำงานเสร็จครบหมด
27     for p in processes:
28         p.join()
29
30     # 3. แสดงผลลัพธ์ภาพรวม
31     duration = time() - start_time
32     print(f"{ctime()} สำเร็จ! ทุกตู้ทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด:
33     {duration:0.2f} วินาที")
34
35 # สิ่งสำคัญที่สุดสำหรับ Multi-processing ใน Python: ต้องครอบด้วยบล็อกนี้เสมอ
36 if __name__ == "__main__":
37     main()
```

Multi-Processing Coffee

```
Sun Jun 21 22:43:00 2026 === เริ่มระบบจำลองตุ๊กตาแฟแบบ Multi-processing ===
Sun Jun 21 22:43:00 2026 [Process ID: 17734] กำลังชงกาแฟให้ ลูกค้า A...
Sun Jun 21 22:43:00 2026 [Process ID: 17736] กำลังชงกาแฟให้ ลูกค้า C...
Sun Jun 21 22:43:00 2026 [Process ID: 17735] กำลังชงกาแฟให้ ลูกค้า B...
Sun Jun 21 22:43:01 2026 [Process ID: 17734] ลูกค้า A: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17736] ลูกค้า C: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17735] ลูกค้า B: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 สำเร็จ! ทุกตูทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด: 1.07 วินาที
```

Asyncio Coffee

- เมื่อเปลี่ยนแนวคิดมาเป็น asyncio (Asynchronous I/O) ในเชิงเปรียบเทียบกับการวาดภาพ หรือการชงกาแฟ จะมีความแตกต่างจาก Threading และ Multiprocessing อย่างสิ้นเชิง
- แนวคิดนี้ไม่ใช้การเพิ่มช่างวาดภาพ หรือเพิ่มตู้กาแฟ แต่คือ "การใช้ตู้กาแฟตู้เดิม 1 ตู้ (หรือช่างคนเดิม 1 คน) ที่บริหารเวลาได้อย่างชาญฉลาด" ในโจทย์เดิม ตู้กาแฟต้องยืนนิ่งๆ รอให้น้ำร้อนเดือด 1 นาทีถึงจะทำขั้นตอนต่อไปได้ (Synchronous) แต่พอเป็น asyncio ตู้กาแฟจะเปลี่ยนพฤติกรรม
- พอสั่งต้มน้ำร้อนของลูกค้า A ปุ่ม ในระหว่างที่ "รอ" น้ำเดือด ตู้จะหันไปกดรับออเดอร์และเริ่มเตรียมแก้วให้ลูกค้า B และ C ทันที โดยสลับการทำงานไปมาในตู้เดียว ทำให้ไม่ต้องเสียเวลารอเปล่าๆ ครับ

```
1 import asyncio
2 # define a coroutine
3 ✓ async def custom_coro():
4     # await another coroutine
5     await asyncio.sleep(1)
6
7 # main coroutine
8 ✓ async def main():
9     # execute my custom coroutine
10    await custom_coro()
11
12 # start the coroutine program
13 asyncio.run(main())
```

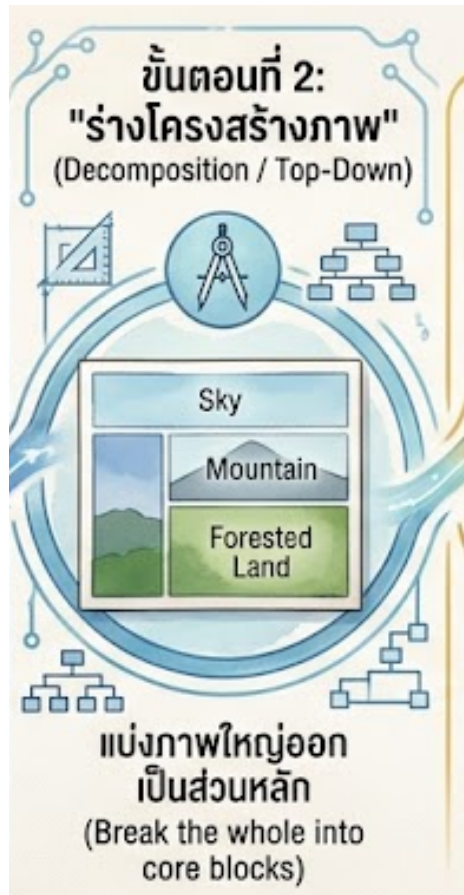
ขั้นตอนที่ 1: มองเห็นภาพรวม (Requirement)



- “เป้าหมาย”
- การคิดเชิงเปรียบเทียบ: ภาพของตู้กาแฟอัจฉริยะ 1 ตู้ ที่สามารถสลับไปทำงานอื่นได้ทันทีเมื่อเจอขั้นตอนที่ต้อง "รอ" (Non-blocking I/O)
- เป้าหมายของระบบ: ทำงานแบบ Single-thread (ใช้ CPU Core เดียว ตัวเดียว) แต่สามารถทำภารกิจของทุกคนให้เสร็จพร้อมๆ กันได้ในเวลาประมาณ 1 วินาทีเท่าเดิม

ขั้นตอนที่ 2: ร่างโครงสร้างภาพ (Decomposition / Top-Down)

แตก โปรแกรมออกเป็นส่วนย่อย ๆ



- เปลี่ยนฟังก์ชันธรรมดาให้เป็น Coroutine: ฟังก์ชันซิงกาแพต้องประกาศด้วยคำว่า **async def** เพื่อให้สามารถสลับงานได้
- จุดเปลี่ยนผ่านเวลาจำลอง: ใช้ **await asyncio.sleep(1)** แทนการ sleep ปกติ เพื่อบอกระบบว่า "ช่วงนี้ฉันกำลังรอนะ ตู่ว่าง ไปทำงานอื่นก่อนได้เลย"
- รวบรวมงาน (Task Gathering): ใช้ **asyncio.gather** เพื่อมัดรวมออเดอร์ทของทุกคนแล้ว โยนให้ Event Loop บริหาร

ขั้นตอนที่ 3: ลงรายละเอียดทีละส่วน (Implementation / Bottom-Up)



- ส่วนที่ 1: ปรับฟังก์ชันชงกาแฟ
- ส่วนที่ 2: เปลี่ยนคำสั่งสร้างหน่วยประมวลผล

```
# ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
def make_coffee(customer_name):
    print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]  
กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
    sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที
    print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]  
ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
```

```
# 1. แยกกระบวนการทำงานแยกจากกันเป็นอิสระ (คนละตัว คนละ CPU Core)
for customer in queue:
    p = multiprocessing.Process(target=make_coffee, args=(customer,))
    processes.append(p)
    p.start() # สั่งเริ่มประมวลผลทันที

# 2. รอจนกระทั่งทุก Process ทำงานเสร็จครบหมด
for p in processes:
    p.join()
```


ขั้นตอนที่ 4: ตรวจสอบ (Testing / Debugging)



- การตรวจสอบใน Multi-processing ต้องระวังเรื่องสำคัญ คือ Process ID (PID) และหน่วยความจำที่แยกจากกัน:
- ตรวจสอบว่าใน Task Manager หรือ Activity Monitor มีกระบวนการ Python ก่อขึ้นขึ้นมาใหม่ 3 อันจริงไหม?
- ข้อมูลที่พิมพ์ออกมาทางหน้าจอต้องไม่ตีกัน (ซึ่งบางระบบปฏิบัติการอาจจะแสดงผลสลับบรรทัดกันเล็กน้อยเนื่องจากส่งข้อมูลมาจากคนละหน่วยความจำ)

```
Sun Jun 21 22:43:00 2026 === เริ่มระบบจำลองดูกาแฟแบบ Multi-processing ===
Sun Jun 21 22:43:00 2026 [Process ID: 17734] กำลังชงกาแฟให้ ลูกค้า A...
Sun Jun 21 22:43:00 2026 [Process ID: 17736] กำลังชงกาแฟให้ ลูกค้า C...
Sun Jun 21 22:43:00 2026 [Process ID: 17735] กำลังชงกาแฟให้ ลูกค้า B...
Sun Jun 21 22:43:01 2026 [Process ID: 17734] ลูกค้า A: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17736] ลูกค้า C: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17735] ลูกค้า B: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 สำเร็จ! ทุกคู่ทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด: 1.07 วินาที
```

ขั้นตอนที่ 5: "ประกอบภาพและเก็บรายละเอียด" (Integration & Refactoring)



- ประกอบชิ้นส่วน

```
1 from time import sleep, ctime, time
2 import multiprocessing
3
4 # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
5 def make_coffee(customer_name):
6     print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]
7     กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
8     sleep(1) # จำลองเวลาชงกาแฟ 1 วินาที
9     print(f"{ctime()} [Process ID: {multiprocessing.current_process().pid}]
10    ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
11
12 def main():
13     # คิวลูกค้า
14     queue = ['A', 'B', 'C']
15     print(f"{ctime()} === เริ่มระบบจำลองตู้กาแฟแบบ Multi-processing ===")
16     start_time = time()
17
18     processes = [] # กระดานเก็บรายชื่อหน่วยประมวลผล (Processes)
19
20     # 1. แดกกระบวนการทำงานแยกจากกันเป็นอิสระ (คนละตู้ คนละ CPU Core)
21     for customer in queue:
22         p = multiprocessing.Process(target=make_coffee, args=(customer,))
23         processes.append(p)
24         p.start() # สั่งเริ่มประมวลผลทันที
25
26     # 2. รอจนกระทั่งทุก Process ทำงานเสร็จครบหมด
27     for p in processes:
28         p.join()
29
30     # 3. แสดงผลลัพธ์ภาพรวม
31     duration = time() - start_time
32     print(f"{ctime()} สำเร็จ! ทุกตู้ทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด:
33     {duration:0.2f} วินาที")
34
35 # สิ่งสำคัญที่สุดสำหรับ Multi-processing ใน Python: ต้องครอบด้วยบล็อกนี้เสมอ
36 if __name__ == "__main__":
37     main()
```

Multi-Processing Coffee

```
Sun Jun 21 22:43:00 2026 === เริ่มระบบจำลองตุกกาแฟแบบ Multi-processing ===
Sun Jun 21 22:43:00 2026 [Process ID: 17734] กำลังชงกาแฟให้ ลูกค้า A...
Sun Jun 21 22:43:00 2026 [Process ID: 17736] กำลังชงกาแฟให้ ลูกค้า C...
Sun Jun 21 22:43:00 2026 [Process ID: 17735] กำลังชงกาแฟให้ ลูกค้า B...
Sun Jun 21 22:43:01 2026 [Process ID: 17734] ลูกค้า A: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17736] ลูกค้า C: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 [Process ID: 17735] ลูกค้า B: ได้รับกาแฟแล้ว!
Sun Jun 21 22:43:01 2026 สำเร็จ! ทุกตู้ทำงานแยกกันเสร็จสิ้น ใช้เวลารวมทั้งหมด: 1.07 วินาที
```

Process ID, Thread ID/Name

- Mac => top -l 0 | grep -i "ชื่อโปรเซส"

-

```
while true; do clear; echo "PID  TH  %CPU  MEM(MB)  STAT  TIME  COMMAND";  
ps -ax -o pid,%cpu,rss,state,time,command | grep -i "[p]ython" | while read -r pid cpu  
rss stat time cmd; do th_count=$(ps -M "$pid" 2>/dev/null | grep -v "USER" | wc -l | tr  
-d ' '); if [ -z "$th_count" ] || [ "$th_count" -eq 0 ]; then continue; fi; mem_mb=$(awk -v  
r="$rss" 'BEGIN {printf "%.1f", r/1024}'); if [ ${#cmd} -gt 50 ]; then cmd="$  
{cmd:0:47}..."; fi; printf "%-6s %-4s %-6s %-9s %-6s %-8s %s\n" "$pid" "$th_count"  
"$cpu" "$mem_mb" "$stat" "$time" "$cmd"; done; sleep 1; done
```

Process ID, Thread ID/Name

- Windows =>

```
while($true) {
    Clear-Host
    Get-CimInstance Win32_Process -Filter "Name LIKE '%python%'" | ForEach-Object {
        $proc = $_
        $perf = Get-Process -Id $proc.ProcessId -ErrorAction SilentlyContinue

        if ($perf) {
            # ลูปเจาะลึกเข้าไปในทุกๆ Thread ของ Process นี้
            foreach ($thread in $perf.Threads) {
                [PSCustomObject]@{
                    PID      = $proc.ProcessId
                    TID      = $thread.Id          # แสดงรหัส Thread ID แยกรายตัว
                    TState   = $thread.ThreadState # สถานะของเธรดนั้น (Running/Waiting)
                    "CPU(s)" = if ($perf.CPU) { [Math]::Round($perf.CPU, 2) } else { 0 }
                    "Mem(MB)" = [Math]::Round(($proc.WorkingSetSize / 1MB), 2)
                    PStat   = if ($perf.Responding) { "Running" } else { "Not Responding" }
                    Time    = (New-TimeSpan -Start $proc.CreationDate -End (Get-Date)).ToString("hh:mm:ss")
                    Command = $proc.CommandLine
                }
            }
        }
    } | Format-Table -AutoSize
    Start-Sleep -Seconds 2
}
```

Process ID, Thread ID/Name

- ตัวเลข ThreadCount ที่นับได้จากฝั่ง Windows มักจะ มากกว่า จำนวนเธรดที่ เขียนใน โค้ด Python เสมอ เช่น
 - ใน โค้ดสั่งรัน 1 เธรดหลัก แต่ Windows นับได้ 3-5 เธรด
 - เนื่องจาก Windows kernel และตัว Python interpreter จะต้องสร้างเธรดภายในขึ้นมาช่วยจัดการระบบ
 - เธรดสำหรับ Garbage Collection
 - เธรดรอรับสัญญาณ (Signal Handling)
 - หรือไอ โอ โพรเซส (I/O Profiles)

Process ID, Thread ID/Name

pid01-synchronous.py

```

1  from time import sleep, ctime, time
2  import os
3  import threading
4
5  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คนแบบซิงโครนัส
6  def make_coffee(customer_name):
7      # ดึง PID และ Thread ID ออกมาดู
8      pid = os.getpid()
9      thread_id = threading.current_thread().native_id
10     thread_name = threading.current_thread().name
11
12     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
13     sleep(5) # บล็อกการทำงานของ Thread ไว้ 5 วินาทีเต็ม
14     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
15
16  def main():
17     queue = ['A', 'B', 'C']
18     main_pid = os.getpid()
19     main_tid = threading.current_thread().native_id
20
21     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] == เริ่มระบบจำลองตู้กาแฟแบบ Synchronous ==")
22     start_time = time()
23
24     # ลูปทำงานตามลำดับคิวเดียว (ทีละคน)
25     for customer in queue:
26         make_coffee(customer)
27
28     duration = time() - start_time
29     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
30
31  if __name__ == "__main__":
32     main()

```


Process ID, Thread ID/Name

pid01-synchronous.py

```
Mon Jun 22 18:30:53 2026 | [Main PID: 11389] [Main TID: 785934] === เริ่มระบบจำลองตู้กาแฟแบบ Synchronous ===
Mon Jun 22 18:30:53 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า A...
Mon Jun 22 18:30:58 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] ลูกค้า A: ได้รับกาแฟแล้ว!
Mon Jun 22 18:30:58 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า B...
Mon Jun 22 18:31:03 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] ลูกค้า B: ได้รับกาแฟแล้ว!
Mon Jun 22 18:31:03 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า C...
Mon Jun 22 18:31:08 2026 | [PID: 11389] [TID: 785934] [Thread Name: MainThread] ลูกค้า C: ได้รับกาแฟแล้ว!
Mon Jun 22 18:31:08 2026 | ใช้เวลารวมทั้งหมด: 15.01 วินาที
```

```
1  from time import sleep, ctime, time
2  import os
3  import threading
4
5  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คนแบบซิงโครนัส
6  def make_coffee(customer_name):
7      # ดึง PID และ Thread ID ออกมาดู
8      pid = os.getpid()
9      thread_id = threading.current_thread().native_id
10     thread_name = threading.current_thread().name
11
12     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
13     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
14     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
15
16  def main():
17     queue = ['A', 'B', 'C']
18     main_pid = os.getpid()
19     main_tid = threading.current_thread().native_id
20
21     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Synchronous ===")
22     start_time = time()
23
24     # ลูปทำงานตามลำดับคิวเดียว (ทีละคน)
25     for customer in queue:
26         make_coffee(customer)
27
28     duration = time() - start_time
29     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
30
31  if __name__ == "__main__":
32     main()
```

Process ID, Thread ID/Name

pid02-thread.py

```

1  from time import sleep, ctime, time
2  import threading
3  import os
4
5  # ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
6  def make_coffee(customer_name):
7      # ดึง PID ของระบบ (จะเหมือนกันทุก Thread)
8      pid = os.getpid()
9      thread_id = threading.current_thread().native_id
10     thread_name = threading.current_thread().name
11
12     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
13     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
14     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
15
16 def main():
17     queue = ['A', 'B', 'C']
18     main_pid = os.getpid()
19     main_tid = threading.current_thread().native_id
20
21     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองชงกาแฟแบบ Multi-Thread ===")
22     start_time = time()
23
24     threads = []
25     # ลูปการทำงาน Thread
26     for customer in queue:
27         # เราสามารถตั้งชื่อ Thread ผ่าน keyword name= ได้เพื่อให้ไล่ได้ง่ายขึ้น
28         t = threading.Thread(target=make_coffee, args=(customer,), name=f"Thread-{customer}")
29         threads.append(t)
30         t.start()
31
32     for t in threads:
33         t.join()
34
35     duration = time() - start_time
36     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
37
38 if __name__ == "__main__":
39     main()

```

Process ID, Thread ID/Name

pid02-thread.py

```
Mon Jun 22 18:38:43 2026 | [Main PID: 12578] [Main TID: 795044] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-Thread ===
Mon Jun 22 18:38:43 2026 | [PID: 12578] [TID: 795045] [Thread Name: Thread-A] กำลังชงกาแฟให้ ลูกค้า A...
Mon Jun 22 18:38:43 2026 | [PID: 12578] [TID: 795046] [Thread Name: Thread-B] กำลังชงกาแฟให้ ลูกค้า B...
Mon Jun 22 18:38:43 2026 | [PID: 12578] [TID: 795047] [Thread Name: Thread-C] กำลังชงกาแฟให้ ลูกค้า C...
Mon Jun 22 18:38:48 2026 | [PID: 12578] [TID: 795046] [Thread Name: Thread-B] ลูกค้า B: ได้รับกาแฟแล้ว!
Mon Jun 22 18:38:48 2026 | [PID: 12578] [TID: 795045] [Thread Name: Thread-A] ลูกค้า A: ได้รับกาแฟแล้ว!
Mon Jun 22 18:38:48 2026 | [PID: 12578] [TID: 795047] [Thread Name: Thread-C] ลูกค้า C: ได้รับกาแฟแล้ว!
Mon Jun 22 18:38:48 2026 | ใช้เวลารวมทั้งหมด: 5.01 วินาที
```

```

1  from time import sleep, ctime, time
2  import threading
3  import os
4
5  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
6  def make_coffee(customer_name):
7      # ดึง PID ของระบบ (จะเหมือนกันทุก Thread)
8      pid = os.getpid()
9      thread_id = threading.current_thread().native_id
10     thread_name = threading.current_thread().name
11
12     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
13     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
14     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
15
16 def main():
17     queue = ['A', 'B', 'C']
18     main_pid = os.getpid()
19     main_tid = threading.current_thread().native_id
20
21     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-Thread ===")
22     start_time = time()
23
24     threads = []
25     # ลูปการทำงาน Thread
26     for customer in queue:
27         # เราสามารถตั้งชื่อ Thread ผ่านพารามิเตอร์ name= ได้เพื่อให้ไล่ได้ง่ายขึ้น
28         t = threading.Thread(target=make_coffee, args=(customer,), name=f"Thread-{customer}")
29         threads.append(t)
30         t.start()
31
32     for t in threads:
33         t.join()
34
35     duration = time() - start_time
36     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
37
38 if __name__ == "__main__":
39     main()

```

Process ID, Thread ID/Name

pid03-multiprocess.py

```

1  from time import sleep, ctime, time
2  import multiprocessing
3  import threading
4  import os
5
6  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
7  def make_coffee(customer_name):
8      # ดึง PID ของหน่วยประมวลผลนี้ (ซึ่งจะแยกกันเด็ดขาด)
9      pid = os.getpid()
10     thread_id = threading.current_thread().native_id
11     thread_name = threading.current_thread().name
12
13     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
14     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
15     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
16
17 def main():
18     queue = ['A', 'B', 'C']
19     main_pid = os.getpid()
20     main_tid = threading.current_thread().native_id
21
22     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-processing ===")
23     start_time = time()
24
25     processes = []
26     # ลูปการทำงาน process
27     for customer in queue:
28         # สร้าง Process ใหม่แยกจากกันเพื่อทำกาแฟ
29         p = multiprocessing.Process(target=make_coffee, args=(customer,))
30         processes.append(p)
31         p.start()
32
33     for p in processes:
34         p.join()
35
36     duration = time() - start_time
37     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
38
39 if __name__ == "__main__":
40     main()

```


Process ID, Thread ID/Name

pid03-multiprocess.py

```
Mon Jun 22 18:59:39 2026 | [Main PID: 13797] [Main TID: 813605] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-processing ===
Mon Jun 22 18:59:39 2026 | [PID: 13801] [TID: 813617] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า C...
Mon Jun 22 18:59:39 2026 | [PID: 13800] [TID: 813616] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า B...
Mon Jun 22 18:59:39 2026 | [PID: 13799] [TID: 813615] [Thread Name: MainThread] กำลังชงกาแฟให้ ลูกค้า A...
Mon Jun 22 18:59:44 2026 | [PID: 13800] [TID: 813616] [Thread Name: MainThread] ลูกค้า B: ได้รับกาแฟแล้ว!
Mon Jun 22 18:59:44 2026 | [PID: 13799] [TID: 813615] [Thread Name: MainThread] ลูกค้า A: ได้รับกาแฟแล้ว!
Mon Jun 22 18:59:44 2026 | [PID: 13801] [TID: 813617] [Thread Name: MainThread] ลูกค้า C: ได้รับกาแฟแล้ว!
Mon Jun 22 18:59:44 2026 | ใช้เวลารวมทั้งหมด: 5.06 วินาที
```

```

1  from time import sleep, ctime, time
2  import multiprocessing
3  import threading
4  import os
5
6  # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คน
7  def make_coffee(customer_name):
8      # ดึง PID ของหน่วยประมวลผลนี้ (ซึ่งจะแยกกันเด็ดขาด)
9      pid = os.getpid()
10     thread_id = threading.current_thread().native_id
11     thread_name = threading.current_thread().name
12
13     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
14     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
15     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
16
17  def main():
18     queue = ['A', 'B', 'C']
19     main_pid = os.getpid()
20     main_tid = threading.current_thread().native_id
21
22     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-processing ===")
23     start_time = time()
24
25     processes = []
26     # ลูปการทำงาน process
27     for customer in queue:
28         # สร้าง Process ใหม่แยกจากกันเด็ดขาด
29         p = multiprocessing.Process(target=make_coffee, args=(customer,))
30         processes.append(p)
31         p.start()
32
33     for p in processes:
34         p.join()
35
36     duration = time() - start_time
37     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
38
39  if __name__ == "__main__":
40     main()

```

Process ID, Thread ID/Name

pid04-asyncio.py

```

1  from time import ctime, time
2  import asyncio
3  import os
4  import threading
5
6  # ฟังก์ชันจำลองการทำงานแบบ Asynchronous
7  async def make_coffee(customer_name):
8      # 1. ดู Process ID และ Thread ID (ซึ่งจะพบว่าเหมือนกันทุกตัว)
9      pid = os.getpid()
10     thread_id = threading.current_thread().native_id
11
12     # 2. ดูข้อมูล Task ปัจจุบันของ asyncio
13     current_task = asyncio.current_task()
14     task_name = current_task.get_name() # ชื่อ Task
15
16     # ใน Python 3.12+ สามารถใช้ Unique ID ของ Task ได้ผ่าน id(current_task)
17     task_id = id(current_task)
18
19     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
20     # จุดสลับงาน (Non-blocking wait)
21     await asyncio.sleep(5)
22     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
23
24  async def main():
25     queue = ['A', 'B', 'C']
26     main_pid = os.getpid()
27     main_tid = threading.current_thread().native_id
28
29     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] == เริ่มระบบจำลองตู้กาแฟแบบ asyncio ==")
30     start_time = time()
31
32     tasks = []
33     for customer in queue:
34         # สร้าง Coroutine
35         coro = make_coffee(customer)
36         # แปลง Coroutine ให้เป็น Task
37         task = asyncio.create_task(coro, name=f"Task-{customer}")
38         tasks.append(task)
39
40     # สั่งให้ทำงานพร้อมกัน
41     await asyncio.gather(*tasks)
42
43     duration = time() - start_time
44     print(f"{ctime()} | ใช้เวลาทั้งหมด: {duration:0.2f} วินาที")
45
46  if __name__ == "__main__":
47     asyncio.run(main())

```

ใช้ Event Loop บริหาร และสั่งให้

Process ID, Thread ID/Name

pid04-asyncio.py

```
Mon Jun 22 19:55:27 2026 | [Main PID: 16290] [Main TID: 853718] === เริ่มระบบจำลองตู้กาแฟแบบ asyncio ===
Mon Jun 22 19:55:27 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456121840] [Task Name: Task-A] กำลังชงกาแฟให้ ลูกค้า A...
Mon Jun 22 19:55:27 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456071248] [Task Name: Task-B] กำลังชงกาแฟให้ ลูกค้า B...
Mon Jun 22 19:55:27 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456072176] [Task Name: Task-C] กำลังชงกาแฟให้ ลูกค้า C...
Mon Jun 22 19:55:32 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456121840] [Task Name: Task-A] ลูกค้า A: ได้รับกาแฟแล้ว!
Mon Jun 22 19:55:32 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456071248] [Task Name: Task-B] ลูกค้า B: ได้รับกาแฟแล้ว!
Mon Jun 22 19:55:32 2026 | [PID: 16290] [TID: 853718] [Async Task ID: 4456072176] [Task Name: Task-C] ลูกค้า C: ได้รับกาแฟแล้ว!
Mon Jun 22 19:55:32 2026 | ใช้เวลารวมทั้งหมด: 5.00 วินาที
```

```

1  from time import ctime, time
2  import asyncio
3  import os
4  import threading
5
6  # ฟังก์ชันจำลองการทำงานแบบ Asynchronous
7  async def make_coffee(customer_name):
8      # 1. ดู Process ID และ Thread ID (ซึ่งจะพบว่าเหมือนกันทุกตัว)
9      pid = os.getpid()
10     thread_id = threading.current_thread().native_id
11
12     # 2. ดูข้อมูล Task ปัจจุบันของ asyncio
13     current_task = asyncio.current_task()
14     task_name = current_task.get_name() # ชื่อ Task
15
16     # ใน Python 3.12+ สามารถใช้ Unique ID ของ Task ได้ผ่าน id(current_task)
17     task_id = id(current_task)
18
19     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
20     # จุดสลับงาน (Non-blocking wait)
21     await asyncio.sleep(5)
22     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
23
24  async def main():
25     queue = ['A', 'B', 'C']
26     main_pid = os.getpid()
27     main_tid = threading.current_thread().native_id
28
29     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ asyncio ===")
30     start_time = time()
31
32     tasks = []
33     for customer in queue:
34         # สร้าง Coroutine
35         coro = make_coffee(customer)
36         # แปลง Coroutine ให้เป็น Task เพื่อให้ Event Loop บริหาร และตั้งชื่อได้
37         task = asyncio.create_task(coro, name=f"Task-{customer}")
38         tasks.append(task)
39
40     # สั่งให้ทำงานพร้อมกัน
41     await asyncio.gather(*tasks)
42
43     duration = time() - start_time
44     print(f"{ctime()} | ใช้เวลารวมทั้งหมด: {duration:0.2f} วินาที")
45
46  if __name__ == "__main__":
47     asyncio.run(main())

```

Resources Consumption

- psutil (Process and System Utilities) module to get resources consumption.
 - CPU time
 - Memory consumption

Synchronous Resource Consumption

ps01-synchronous.py

```
1 from time import sleep, ctime, time, process_time
2 import os
3 import threading
4 import psutil
5
6
7 # ฟังก์ชันจำลองการทำกาแฟให้ลูกค้า 1 คนแบบซิงโครนัส
8 def make_coffee(customer_name):
9     # ดึง PID และ Thread ID ออกมาดู
10    pid = os.getpid()
11    thread_id = threading.current_thread().native_id
12    thread_name = threading.current_thread().name
13
14    print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
15    sum(i * i for i in range(1000000)) # จำลองงานคำนวณ (CPU-bound) เล็กน้อย และรอ 5 วินาที
16    sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
17    print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
18
19 def main():
20    queue = ['A', 'B', 'C']
21    main_pid = os.getpid()
22    main_tid = threading.current_thread().native_id
23
24    print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Synchronous ===")
25    start_time = time()
26    start_cpu = process_time() # เริ่มจับเวลา CPU
27
28    # ทำงานตามลำดับคิวเดียว (ทีละคน)
29    for customer in queue:
30        make_coffee(customer)
31
32    duration = time() - start_time
33    cpu_duration = process_time() - start_cpu
34
35    # ใช้ psutil ดึงค่าการกิน RAM
36    process = psutil.Process(os.getpid())
37    mem_mb = process.memory_info().rss / (1024 * 1024)
38
39    print(f"[สรุปผล Synchronous]")
40    print(f"เวลาที่ใช้จริง (Wall Time): {duration:0.2f} วินาที")
41    print(f"เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): {cpu_duration:0.4f} วินาที")
42    print(f"ทรัพยากร Memory (RAM) ที่ใช้: {mem_mb:.2f} MB")
43
44 if __name__ == "__main__":
45    main()
```


Multi-thread Resource Consumption

ps02-thread.py

```
1  from time import sleep, ctime, time, process_time
2  import threading
3  import os
4  import psutil
5
6  # ฟังก์ชันจำลองการทำการเทพให้ลูกค้า 1 คน
7  def make_coffee(customer_name):
8      # ตั้ง PID ของระบบ (จะเหมือนกันทุก Thread)
9      pid = os.getpid()
10     thread_id = threading.current_thread().native_id
11     thread_name = threading.current_thread().name
12
13     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ลูกค้า {customer_name}...")
14     sum(i * i for i in range(1000000)) # จำลองงานคำนวณ (CPU-bound) เล็กน้อย และรอ 5 วินาที
15     sleep(5) # นอนพักการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
16     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
17
18 def main():
19     queue = ['A', 'B', 'C']
20     main_pid = os.getpid()
21     main_tid = threading.current_thread().native_id
22
23     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ Multi-Thread ===")
24     start_time = time()
25     start_cpu = process_time() # เริ่มจับเวลา CPU
26
27     threads = []
28     # อุปการะงาน Thread
29     for customer in queue:
30         # เราสามารถตั้งชื่อ Thread ผ่านพารามิเตอร์ name= ได้เพื่อให้ได้ชัดเจนขึ้น
31         t = threading.Thread(target=make_coffee, args=(customer,), name=f"Thread-{customer}")
32         threads.append(t)
33         t.start()
34
35     for t in threads:
36         t.join()
37
38     duration = time() - start_time
39     cpu_duration = process_time() - start_cpu
40
41     # ใช้ psutil ตรวจสอบการใช้ RAM
42     process = psutil.Process(os.getpid())
43     mem_mb = process.memory_info().rss / (1024 * 1024)
44
45     print(f"[สรุปผล Multi-Thread]")
46     print(f"เวลาที่ใช้จริง (Wall Time): {duration:0.2f} วินาที")
47     print(f"เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): {cpu_duration:0.4f} วินาที")
48     print(f"ทรัพยากร Memory (RAM) ที่ใช้: {mem_mb:.2f} MB")
49
50 if __name__ == "__main__":
51     main()
```

Multi-process Resources

ps03-multiprocess.py

```
1  from time import sleep, ctime, time, process_time
2  import multiprocessing
3  import threading
4  import os
5  import psutil
6
7  # ฟังก์ชันจำลองการชงกาแฟให้ลูกค้า 1 คน
8  def make_coffee(customer_name, result_queue):
9      # ดึง PID ของหน่วยประมวลผลนี้ (ซึ่งจะแยกกันเด็ดขาด)
10     pid = os.getpid()
11     thread_id = threading.current_thread().native_id
12     thread_name = threading.current_thread().name
13
14     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
15     start_cpu = process_time()
16     sum(i * i for i in range(1000000)) # จำลองงานคำนวณ (CPU-bound) เล็กน้อย และรอ 5 วินาที
17     sleep(5) # บล็อกการทำงานของ Thread นี้ไว้ 5 วินาทีเต็มๆ
18     cpu_duration = process_time() - start_cpu
19     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Thread Name: {thread_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
20
21     # ส่งค่าการกิน RAM และ CPU ของตัวเองกลับไปให้หน่วยหลักผ่าน Queue
22     process = psutil.Process(pid)
23     mem_mb = process.memory_info().rss / (1024 * 1024)
24     result_queue.put((mem_mb, cpu_duration))
25
```

Multi-process Resource Consumption

ps03-multiprocess.py

```
26 def main():
27     queue = ['A', 'B', 'C']
28     main_pid = os.getpid()
29     main_tid = threading.current_thread().native_id
30
31     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองสถานการณ์ Multi-processing ===")
32     start_time = time()
33     main_start_cpu = process_time()
34
35     result_queue = multiprocessing.Queue()
36     processes = []
37     # ลูปการทำงาน process
38     for customer in queue:
39         # สร้าง Process ใหม่แยกจากกันเพื่อทดสอบ
40         p = multiprocessing.Process(target=make_coffee, args=(customer, result_queue))
41         processes.append(p)
42         p.start()
43
44     # รวบรวมข้อมูลทรัพยากรจากทุก Process ย่อย
45     child_memories = []
46     child_cpu_times = []
47     for _ in queue:
48         mem, cpu_t = result_queue.get()
49         child_memories.append(mem)
50         child_cpu_times.append(cpu_t)
51
52     for p in processes:
53         p.join()
54
55     duration = time() - start_time
56
57     # คำนวณแรมของ Main Process เองด้วย
58     main_process = psutil.Process(os.getpid())
59     main_mem = main_process.memory_info().rss / (1024 * 1024)
60
61     total_memory = main_mem + sum(child_memories)
62     total_cpu_time = (process_time() - main_start_cpu) + sum(child_cpu_times)
63
64     print(f"[สรุปผล Multi-processing]")
65     print(f"เวลาที่ใช้จริง (Wall Time): {duration:0.2f} วินาที")
66     print(f"เวลารวมที่ CPU ทุก Core ช่วยกันประมวลผล (Total CPU Time): {total_cpu_time:0.4f} วินาที")
67     print(f"ทรัพยากร Memory (RAM) รวมทุก Process: {total_memory:.2f} MB (Main: {main_mem:.2f} MB + ย่อย: {sum(child_memories):.2f} MB)")
68
69 if __name__ == "__main__":
70     main()
```

Asyncio Resource Consumption

ps04-asyncio.py

```
1  from time import ctime, time, process_time
2  import asyncio
3  import os
4  import threading
5  import psutil
6
7  # ฟังก์ชันจำลองการทำการกาแฟแบบ Asynchronous
8  async def make_coffee(customer_name):
9      # 1. ดู Process ID และ Thread ID (ซึ่งจะพบว่าเหมือนกันทุกตัว)
10     pid = os.getpid()
11     thread_id = threading.current_thread().native_id
12
13     # 2. ดูข้อมูล Task ปัจจุบันของ asyncio
14     current_task = asyncio.current_task()
15     task_name = current_task.get_name() # ชื่อ Task
16
17     # ใน Python 3.12+ สามารถใช้ Unique ID ของ Task ได้ผ่าน id(current_task)
18     task_id = id(current_task)
19
20     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] กำลังชงกาแฟให้ ลูกค้า {customer_name}...")
21     sum(i * i for i in range(1000000)) # จำลองงานคำนวณ (CPU-bound) เล็กน้อย และรอ 5 วินาที
22     # จุดสลับงาน (Non-blocking wait)
23     await asyncio.sleep(5)
24     print(f"{ctime()} | [PID: {pid}] [TID: {thread_id}] [Async Task ID: {task_id}] [Task Name: {task_name}] ลูกค้า {customer_name}: ได้รับกาแฟแล้ว!")
25
26     # ... (code continues)
```

Asyncio Resource Consumption

ps04-asyncio.py

```
26 async def main():
27     queue = ['A', 'B', 'C']
28     main_pid = os.getpid()
29     main_tid = threading.current_thread().native_id
30
31     print(f"{ctime()} | [Main PID: {main_pid}] [Main TID: {main_tid}] === เริ่มระบบจำลองตู้กาแฟแบบ asyncio ===")
32     start_time = time()
33     start_cpu = process_time()
34
35     tasks = []
36     for customer in queue:
37         # สร้าง Coroutine
38         coro = make_coffee(customer)
39         # แปลง Coroutine ให้เป็น Task เพื่อให้ Event Loop บริหาร และตั้งชื่อได้
40         task = asyncio.create_task(coro, name=f"Task-{customer}")
41         tasks.append(task)
42
43     # สั่งให้ทำพร้อมกัน
44     await asyncio.gather(*tasks)
45
46     duration = time() - start_time
47     cpu_duration = process_time() - start_cpu
48
49     process = psutil.Process(os.getpid())
50     mem_mb = process.memory_info().rss / (1024 * 1024)
51
52     print(f"[สรุปผล Asyncio]")
53     print(f"เวลาที่ใช้จริง (Wall Time): {duration:0.2f} วินาที")
54     print(f"เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): {cpu_duration:0.4f} วินาที")
55     print(f"ทรัพยากร Memory (RAM) ที่ใช้: {mem_mb:.2f} MB")
56
57 if __name__ == "__main__":
58     asyncio.run(main())
```

Resource Consumption

```
Mon Jun 22 21:14:33 2026 | [Main PID: 31887] [Main TID: 936404] === เริ่มระบบจำลองดูการแพแบบ Synchronous ===
Mon Jun 22 21:14:33 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา A...
Mon Jun 22 21:14:38 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] ลูกคา A: ได้รับกาแพแล้ว!
Mon Jun 22 21:14:38 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา B...
Mon Jun 22 21:14:43 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] ลูกคา B: ได้รับกาแพแล้ว!
Mon Jun 22 21:14:43 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา C...
Mon Jun 22 21:14:48 2026 | [PID: 31887] [TID: 936404] [Thread Name: MainThread] ลูกคา C: ได้รับกาแพแล้ว!
[สรุปผล Synchronous]
เวลาที่ใช้จริง (Wall Time): 15.18 วินาที
เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): 0.1641 วินาที
ทรัพยากร Memory (RAM) ที่ใช้: 17.36 MB
```

```
Mon Jun 22 21:15:09 2026 | [Main PID: 32260] [Main TID: 937441] === เริ่มระบบจำลองดูการแพแบบ Multi-Thread ===
Mon Jun 22 21:15:09 2026 | [PID: 32260] [TID: 937443] [Thread Name: Thread-A] กำลังซิงกาแพให้ ลูกคา A...
Mon Jun 22 21:15:09 2026 | [PID: 32260] [TID: 937444] [Thread Name: Thread-B] กำลังซิงกาแพให้ ลูกคา B...
Mon Jun 22 21:15:09 2026 | [PID: 32260] [TID: 937445] [Thread Name: Thread-C] กำลังซิงกาแพให้ ลูกคา C...
Mon Jun 22 21:15:14 2026 | [PID: 32260] [TID: 937443] [Thread Name: Thread-A] ลูกคา A: ได้รับกาแพแล้ว!
Mon Jun 22 21:15:14 2026 | [PID: 32260] [TID: 937444] [Thread Name: Thread-B] ลูกคา B: ได้รับกาแพแล้ว!
Mon Jun 22 21:15:14 2026 | [PID: 32260] [TID: 937445] [Thread Name: Thread-C] ลูกคา C: ได้รับกาแพแล้ว!
[สรุปผล Multi-Thread]
เวลาที่ใช้จริง (Wall Time): 5.14 วินาที
เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): 0.1282 วินาที
ทรัพยากร Memory (RAM) ที่ใช้: 17.30 MB
```

```
Mon Jun 22 21:15:33 2026 | [Main PID: 32632] [Main TID: 938283] === เริ่มระบบจำลองดูการแพแบบ Multi-processing ===
Mon Jun 22 21:15:33 2026 | [PID: 32634] [TID: 938293] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา A...
Mon Jun 22 21:15:33 2026 | [PID: 32636] [TID: 938295] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา C...
Mon Jun 22 21:15:33 2026 | [PID: 32635] [TID: 938294] [Thread Name: MainThread] กำลังซิงกาแพให้ ลูกคา B...
Mon Jun 22 21:15:38 2026 | [PID: 32636] [TID: 938295] [Thread Name: MainThread] ลูกคา C: ได้รับกาแพแล้ว!
Mon Jun 22 21:15:38 2026 | [PID: 32634] [TID: 938293] [Thread Name: MainThread] ลูกคา A: ได้รับกาแพแล้ว!
Mon Jun 22 21:15:38 2026 | [PID: 32635] [TID: 938294] [Thread Name: MainThread] ลูกคา B: ได้รับกาแพแล้ว!
[สรุปผล Multi-processing]
เวลาที่ใช้จริง (Wall Time): 5.14 วินาที
เวลารวมที่ CPU ทุก Core ช่วยกันประมวลผล (Total CPU Time): 0.1421 วินาที
ทรัพยากร Memory (RAM) รวมทุก Process: 84.48 MB (Main: 20.89 MB + ย่อย: 63.59 MB)
```

```
Mon Jun 22 21:16:37 2026 | [Main PID: 33011] [Main TID: 939812] === เริ่มระบบจำลองดูการแพแบบ asyncio ===
Mon Jun 22 21:16:37 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387769232] [Task Name: Task-A] กำลังซิงกาแพให้ ลูกคา A...
Mon Jun 22 21:16:37 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387722768] [Task Name: Task-B] กำลังซิงกาแพให้ ลูกคา B...
Mon Jun 22 21:16:37 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387723232] [Task Name: Task-C] กำลังซิงกาแพให้ ลูกคา C...
Mon Jun 22 21:16:42 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387769232] [Task Name: Task-A] ลูกคา A: ได้รับกาแพแล้ว!
Mon Jun 22 21:16:42 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387722768] [Task Name: Task-B] ลูกคา B: ได้รับกาแพแล้ว!
Mon Jun 22 21:16:42 2026 | [PID: 33011] [TID: 939812] [Async Task ID: 4387723232] [Task Name: Task-C] ลูกคา C: ได้รับกาแพแล้ว!
[สรุปผล Asyncio]
เวลาที่ใช้จริง (Wall Time): 5.12 วินาที
เวลาที่ CPU ใช้ประมวลผลจริง (CPU Time): 0.1186 วินาที
ทรัพยากร Memory (RAM) ที่ใช้: 24.34 MB
```

- Memory (RAM) ของ Multi-processing จะโดดเด่นที่สุดอย่างชัดเจน: ตัวเลขรวมสูงสุด เพราะมันประกอบไปด้วย RAM ผังตัวหลัก และ RAM ผังตัวย่อยอีก 3 ตัวแยกกัน ในขณะที่ไฟล์อื่นจะนิ่งอยู่ที่ประมาณ 17-24MB
- Wall Time (เวลาจริง): เวอร์ชัน Synchronous จะลากยาวไปถึง 15 วินาที (ซิง 3 แก้ว แก้วละ 5 วินาทีเรียงกัน) ส่วนอีก 3 ตัวที่เหลือจะทำงานพร้อมกัน/สลับกันทำงาน ทำให้จบงานได้ในเวลาใกล้เคียงกันคือประมาณ 5 วินาทีนิดๆ
- เวลาของ CPU Time: หากงานในฟังก์ชันคำนวณหนักขึ้นเรื่อยๆ Multi-processing จะเป็นตัวเดียวที่สามารถข้ามขีดจำกัดและทำให้โปรแกรมจบได้เร็วที่สุดเมื่อเทียบกับงานคำนวณปริมาณมหาศาล เพราะดึงพลัง CPU ออกมาใช้ได้หลาย Core จริงๆ

Why Async? The Tech Perspective

OVERCOMING ARCHITECTURAL BOTTLENECKS

Breaking the Memory Wall

Traditional threads are heavily resource-intensive for the operating system.

- **Threading:** Allocates a fixed stack (e.g., ~8MB) per thread. 10,000 users can instantly deplete server RAM.
- **Async:** Operates on a single thread. Tasks cost only kilobytes, safely handling massive concurrent connections.

Eliminating OS Overhead

Constantly switching threads drains processor power through background management.

- Bypasses expensive OS **Context Switching** (saving/loading thread states).
- The **Event Loop** handles task swaps lightweightly inside Python, letting the CPU focus purely on execution.

The "Waiting" Dilemma in Modern Systems

THE REALITY OF I/O BOUND TASKS

Maximizing CPU Idling Time

CPUs calculate in nanoseconds, but network operations take millisecond.

- **The Gap:** Waiting 1 second for a web API response is an eternity for a high-speed processor.
- **Synchronous Waste:** Forces the CPU to sit completely idle while waiting for network packets.
- **Asynchronous Rescue:** Commands the CPU to instantly jump to the next client request instead of waiting.



Why Async? The Practical Benefits

REAL-WORLD INDUSTRY APPLICATIONS

High-Speed Modern Web APIs

Industry-standard backend frameworks rely heavily on asynchronous patterns.

- Powering top-tier frameworks like **FastAPI** (Python) and **Node.js** (JavaScript).
- Allows a small, budget-friendly server to handle thousands of API requests smoothly.

Real-Time Systems & Scraping

Essential for data-driven pipelines and live application features

- **Real-Time:** Sustains continuous live feeds like chat servers, live GPS tracking, and stock tickers.
- **Data Gathering:** Fetches 5,000 web pages concurrently in minutes instead of waiting hours sequentially.

Assignment

ระบบจำลองตู้ต้มกาแฟอัจฉริยะ (Coffee Machine Concurrency)

- ตู้กาแฟอัตโนมัติจะต้องบริการลูกค้าทั้งหมด 3 คน คือ A, B และ C ตามลำดับคิว โดยในลูกค้า 1 คน จะต้องทำภารกิจ 2 อย่างต่อเนื่องกัน คือ:
 - กระบวนการชงกาแฟ (Make Coffee): ใช้เวลาจริง 1.0 วินาที
 - กระบวนการแสดงผลจอ LCD (Update LCD Screen): เมื่อชงเสร็จ ต้องส่งข้อมูลไปประมวลผลบนหน้าจอแผงควบคุมตู้ ใช้เวลา 1.0 วินาที
- (รวมเวลาต่อแก้วหากทำงานต่อแผ่นหลังเท่ากับ 2 วินาที)

up01_synchronous.py OUTPUT

```
Mon Jun 22 22:45:12 2026 | === Synchronous Coffee Machine ===
Mon Jun 22 22:45:12 2026 | Making coffee for A...
Mon Jun 22 22:45:13 2026 | Coffee ready for A!
Mon Jun 22 22:45:13 2026 | LCD: Processing for customer A...
Mon Jun 22 22:45:14 2026 | LCD: Done for customer A.
Mon Jun 22 22:45:14 2026 | Making coffee for B...
Mon Jun 22 22:45:15 2026 | Coffee ready for B!
Mon Jun 22 22:45:15 2026 | LCD: Processing for customer B...
Mon Jun 22 22:45:16 2026 | LCD: Done for customer B.
Mon Jun 22 22:45:16 2026 | Making coffee for C...
Mon Jun 22 22:45:17 2026 | Coffee ready for C!
Mon Jun 22 22:45:17 2026 | LCD: Processing for customer C...
Mon Jun 22 22:45:18 2026 | LCD: Done for customer C.
Mon Jun 22 22:45:18 2026 | Total time: 6.02 seconds
```

up02_thread.py OUTPUT

```
Mon Jun 22 22:46:47 2026 | === Multi-threading Coffee Machine ===
Mon Jun 22 22:46:47 2026 | Making coffee for A...
Mon Jun 22 22:46:47 2026 | Making coffee for B...
Mon Jun 22 22:46:47 2026 | Making coffee for C...
Mon Jun 22 22:46:48 2026 | Coffee ready for B!
Mon Jun 22 22:46:48 2026 | Coffee ready for C!
Mon Jun 22 22:46:48 2026 | LCD: Processing for customer B...
Mon Jun 22 22:46:48 2026 | LCD: Processing for customer C...
Mon Jun 22 22:46:48 2026 | Coffee ready for A!
Mon Jun 22 22:46:48 2026 | LCD: Processing for customer A...
Mon Jun 22 22:46:49 2026 | LCD: Done for customer B.
Mon Jun 22 22:46:49 2026 | LCD: Done for customer C.
Mon Jun 22 22:46:49 2026 | LCD: Done for customer A.
Mon Jun 22 22:46:49 2026 | Total time: 2.01 seconds
```

up03_multiprocess.py OUTPUT

```
Mon Jun 22 22:47:44 2026 | === Multi-processing Coffee Machine ===
Mon Jun 22 22:47:44 2026 | Making coffee for A...
Mon Jun 22 22:47:44 2026 | Making coffee for B...
Mon Jun 22 22:47:44 2026 | Making coffee for C...
Mon Jun 22 22:47:45 2026 | Coffee ready for A!
Mon Jun 22 22:47:45 2026 | LCD: Processing for customer A...
Mon Jun 22 22:47:45 2026 | Coffee ready for B!
Mon Jun 22 22:47:45 2026 | LCD: Processing for customer B...
Mon Jun 22 22:47:45 2026 | Coffee ready for C!
Mon Jun 22 22:47:45 2026 | LCD: Processing for customer C...
Mon Jun 22 22:47:46 2026 | LCD: Done for customer A.
Mon Jun 22 22:47:46 2026 | LCD: Done for customer B.
Mon Jun 22 22:47:46 2026 | LCD: Done for customer C.
Mon Jun 22 22:47:46 2026 | Total time: 2.07 seconds
```

up04_asyncio.py OUTPUT

```
Mon Jun 22 22:48:20 2026 | === Asyncio Coffee Machine ===
Mon Jun 22 22:48:20 2026 | Making coffee for A...
Mon Jun 22 22:48:20 2026 | Making coffee for B...
Mon Jun 22 22:48:20 2026 | Making coffee for C...
Mon Jun 22 22:48:21 2026 | Coffee ready for A!
Mon Jun 22 22:48:21 2026 | LCD: Processing for customer A...
Mon Jun 22 22:48:21 2026 | Coffee ready for B!
Mon Jun 22 22:48:21 2026 | LCD: Processing for customer B...
Mon Jun 22 22:48:21 2026 | Coffee ready for C!
Mon Jun 22 22:48:21 2026 | LCD: Processing for customer C...
Mon Jun 22 22:48:22 2026 | LCD: Done for customer A.
Mon Jun 22 22:48:22 2026 | LCD: Done for customer B.
Mon Jun 22 22:48:22 2026 | LCD: Done for customer C.
Mon Jun 22 22:48:22 2026 | Total time: 2.00 seconds
```